

LAPORAN PRAKTIKUM BIG DATA

PRAKTIKUM 1

PENGENALAN BIG DATA DAN LINGKUNGAN KERJA SERTA PENGOLAHAN DATASET BIG DATA MENGUNAKAN PYTHON



Disusun oleh:

Habibi Putra Rizqullah

NIM 2411531001

Kelas A

Dosen Pengampu:

Afdhal Dinilhak, M.Kom, S.Kom

**FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
UNIVERSITAS ANDALAS**

2026

DAFTAR ISI

BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	2
BAB II DASAR TEORI	3
2.1 Big Data dan Karakteristik 5V	3
2.2 Tingkatan Skala Data dan Pemilihan Alat.....	3
2.3 Google Colab sebagai Lingkungan Kerja	4
2.4 pandas.....	4
2.5 CSV dan Parquet	4
2.6 Optimasi Tipe Data.....	5
2.7 Chunking.....	5
2.8 PySpark.....	6
BAB III METODOLOGI PRAKTIKUM.....	7
3.1 Waktu dan Lingkungan Praktikum.....	7
3.2 Perangkat dan Perangkat Lunak	7
3.3 Dataset.....	7
3.4 Akses Notebook dan Repository GitHub.....	8
3.5 Tahapan Praktikum.....	8
BAB IV HASIL DAN PEMBAHASAN.....	9
4.1 Verifikasi Lingkungan Kerja (J-1)	9
4.2 Menghubungkan Google Drive (J-2).....	10
4.3 Menyiapkan Pengukuran Waktu dan Memori (J-3).....	11
4.4 Mengunduh dan Menginspeksi Dataset (J-4)	13
4.5 Memuat Data dan Mengukur Biaya Memori (J-5).....	14
4.6 Pemeriksaan dan Pembersihan Data (J-6).....	17
4.7 Pemrosesan Menggunakan Chunking (J-7)	20
4.8 Pengolahan Menggunakan PySpark (J-8).....	22
4.9 Agregasi dan Penyimpanan Artefak (J-9).....	26
4.10 Visualisasi Hasil (J-10).....	27
4.11 Analisis Hasil Praktikum	29
4.12 Studi Kasus Penjadwalan Armada	31
4.13 Latihan Praktikum.....	35
4.13.1 Lima Nilai trip_distance Terbesar	35
4.13.2 Perbandingan Ukuran Chunk.....	35
4.13.3 Persentase Tip	36
4.13.4 Kompresi Snappy dan Gzip.....	36
4.13.5 Konsistensi pandas dan Spark.....	37
BAB V TUGAS PRAKTIKUM	38
5.1 Dataset Bulan Pilihan.....	38
5.2 Perbandingan Dua Bulan	40
5.3 Laporan Kinerja.....	42
5.4 Refleksi Karakteristik 5V	43
5.5 Reproducibility	44
5.6 Pertanyaan Evaluasi	45
BAB VI PENUTUP.....	48
6.1 Kesimpulan	48
DAFTAR PUSTAKA	51
LAMPIRAN.....	52
Lampiran 1. Tautan Repository dan Notebook.....	52
Lampiran 2. Screenshot Artefak Google Drive.....	52
Lampiran 3. Dokumentasi Error dan Diagnosisnya.....	52
Lampiran 4. Berkas Pendukung.....	52

BAB I

PENDAHULUAN

1.1 Latar Belakang

Volume dan keragaman data yang dihasilkan berbagai aktivitas digital terus meningkat setiap tahun, mulai dari transaksi transportasi, log aplikasi, hingga sensor perangkat. Pertumbuhan ini membuat pendekatan pengolahan data konvensional pada satu mesin dengan kapasitas terbatas mulai menemui hambatan, baik dari sisi kapasitas memori, kecepatan pemrosesan, maupun keragaman format data yang harus ditangani. Data disebut besar bukan semata-mata karena jumlahnya banyak, melainkan karena cara pengolahan konvensional sudah tidak lagi memadai untuk menanganinya secara efisien.

Untuk mengetahui secara pasti kapan sebuah data mulai "terasa besar", diperlukan pengukuran langsung terhadap ukuran data, waktu pemrosesan, dan penggunaan memori, bukan sekadar asumsi. Praktikum ini menggunakan Google Colab sebagai lingkungan kerja karena menyediakan mesin virtual dengan Python dan pustaka analitik yang sudah terpasang tanpa memerlukan instalasi lokal, sehingga mahasiswa dapat langsung fokus pada eksperimen pengolahan data.

Pustaka *pandas* digunakan sebagai alat utama pengolahan data tabular berskala sedang, sedangkan format *Parquet* dimanfaatkan sebagai alternatif format kolumnar yang lebih efisien dibandingkan *CSV*. Ketika ukuran data melampaui kapasitas memori yang tersedia, teknik *chunking* memungkinkan data diproses secara bertahap tanpa memuat seluruh isi file sekaligus. Pada skala yang lebih besar lagi, *PySpark* diperkenalkan sebagai contoh kerangka kerja pemrosesan terdistribusi, meskipun pada dataset berskala sedang seperti yang digunakan dalam praktikum ini, penggunaannya belum tentu memberikan keuntungan performa dibandingkan *pandas*. Keseluruhan rangkaian kegiatan ini secara langsung mempraktikkan karakteristik *Big Data* yang dikenal sebagai 5V, yaitu *Volume*, *Velocity*, *Variety*, *Veracity*, dan *Value*.

1.2 Tujuan Praktikum

1. Memverifikasi lingkungan kerja Google Colab yang digunakan.
2. Mengunduh dan memeriksa integritas dataset publik secara terprogram.
3. Mengukur waktu pemrosesan dan penggunaan memori pada berbagai strategi pengolahan data.
4. Mengurangi penggunaan memori melalui pemilihan kolom dan optimasi tipe data.
5. Memeriksa kualitas data dan membersihkan data dari anomali.
6. Memproses data yang berukuran lebih besar dari memori menggunakan teknik chunking.
7. Membuat tabel agregasi data perjalanan per jam.
8. Membandingkan pendekatan pandas dengan PySpark pada skala data yang sama.
9. Menyimpan hasil pengolahan dalam format CSV dan Parquet.
10. Membuat visualisasi hasil agregasi data.

BAB II

DASAR TEORI

2.1 Big Data dan Karakteristik 5V

Big Data dapat dipahami melalui lima dimensi karakteristik yang dikenal sebagai 5V. Volume merujuk pada ukuran data yang melampaui kapasitas nyaman satu mesin untuk diproses. Velocity merujuk pada kecepatan data dihasilkan dan kebutuhan latensi dalam menghasilkan hasil analisis. Variety merujuk pada keragaman bentuk data, mulai dari data terstruktur, semi-terstruktur, hingga tidak terstruktur. Veracity merujuk pada tingkat kepercayaan dan kualitas data, termasuk keberadaan data yang tidak valid atau anomali. Value merujuk pada nilai atau manfaat bisnis yang dapat dihasilkan dari pengolahan data tersebut.

Pada praktikum ini, dimensi Volume tercermin dari jumlah baris dataset yang mencapai jutaan baris per bulan. Dimensi Veracity tercermin dari keberadaan data anomali seperti jarak perjalanan negatif atau tarif tidak valid yang harus dibersihkan sebelum dianalisis. Dimensi Value tercermin dari hasil agregasi per jam yang dapat langsung dimanfaatkan untuk pengambilan keputusan operasional, seperti dibahas pada studi kasus penjadwalan armada.

2.2 Tingkatan Skala Data dan Pemilihan Alat

Terdapat tiga kondisi umum yang menentukan alat pengolahan data yang tepat digunakan. Pertama, ketika data masih dapat dimuat sepenuhnya ke dalam memori, pandas dan NumPy merupakan pilihan yang paling sederhana dan cepat dikembangkan. Kedua, ketika data lebih besar dari memori namun masih dapat diproses pada satu mesin, teknik seperti chunking, pemilihan kolom, penggunaan format kolumnar seperti Parquet, atau mesin out-of-core seperti Dask dan DuckDB menjadi relevan. Ketiga, ketika data melampaui kapasitas satu mesin, diperlukan pemrosesan terdistribusi seperti Apache Spark atau Hadoop MapReduce yang bekerja di atas penyimpanan terdistribusi.

Perlu ditekankan bahwa Spark tidak selalu lebih cepat dibandingkan pandas. Pada dataset yang masih berukuran kecil hingga sedang, Spark justru dapat lebih

lambat karena adanya overhead penjadwalan tugas, serialisasi antar partisi, dan inisialisasi lingkungan JVM, yang tidak sebanding dengan manfaat pemrosesan paralel pada data sekecil itu. Pemilihan alat yang tepat harus didasarkan pada pengukuran nyata terhadap hubungan antara ukuran data dan sumber daya yang tersedia, bukan berdasarkan asumsi bahwa alat yang lebih kompleks selalu memberikan hasil yang lebih baik.

2.3 Google Colab sebagai Lingkungan Kerja

Google Colab menyediakan mesin virtual berbasis Linux dengan runtime cloud yang telah dilengkapi Python dan berbagai pustaka analitik data tanpa memerlukan proses instalasi lokal. Pada praktikum ini digunakan runtime bertipe CPU karena kebutuhan pemrosesan data tabular tidak memerlukan akselerasi GPU. Konsekuensi penting dari sifat runtime Colab adalah bahwa penyimpanan pada direktori /content bersifat sementara (ephemeral); seluruh berkas pada direktori tersebut akan hilang ketika sesi runtime berakhir atau menjadi idle terlalu lama. Oleh karena itu, seluruh hasil penting seperti berkas agregasi dan visualisasi disimpan ke Google Drive agar tidak hilang. Prinsip reproducibility turut ditekankan, yaitu bahwa notebook harus dapat menyiapkan ulang datanya sendiri dari awal ketika dijalankan pada runtime baru yang bersih.

2.4 pandas

pandas merupakan pustaka Python yang digunakan secara luas untuk pengolahan data tabular. Pada praktikum ini, pandas dimanfaatkan untuk membaca data dari berkas Parquet dan CSV, memilih kolom tertentu yang relevan untuk dianalisis, mengubah tipe data agar penggunaan memori lebih efisien, membersihkan data dari nilai yang tidak valid, serta melakukan agregasi data seperti perhitungan jumlah, rata-rata, dan median per kelompok jam.

2.5 CSV dan Parquet

CSV merupakan format penyimpanan data berbasis baris yang menyimpan setiap nilai sebagai representasi teks tanpa kompresi bawaan. Parquet, sebaliknya, merupakan format penyimpanan kolumnar yang menyimpan data per kolom beserta skema dan kompresinya. Sifat kolumnar Parquet memungkinkan dilakukannya

column pruning, yaitu proses pembacaan yang hanya menyentuh blok kolom yang benar-benar dibutuhkan tanpa harus membaca seluruh baris data terlebih dahulu. Kombinasi antara penyimpanan kolumnar dan kompresi bawaan membuat Parquet secara umum berukuran jauh lebih kecil dan lebih cepat dibaca dibandingkan CSV untuk beban kerja analitik, sebagaimana akan dibuktikan pada Bagian 4.7.

2.6 Optimasi Tipe Data

Secara bawaan, pandas menyimpan data numerik menggunakan tipe `int64` atau `float64` yang masing-masing membutuhkan 8 byte per nilai. Downcasting merupakan teknik mengubah tipe data ke representasi yang lebih ringkas, misalnya dari `float64` menjadi `float32`, tanpa mengubah struktur data secara keseluruhan. Untuk kolom dengan jumlah nilai unik yang sedikit, tipe `category` dapat digunakan untuk menyimpan data sebagai indeks kecil beserta kamus nilainya, sehingga menghemat memori secara signifikan. Perlu diperhatikan bahwa downcasting dari `float64` ke `float32` mengurangi presisi angka desimal; hal ini dapat diterima untuk kebutuhan analisis agregat, tetapi tidak disarankan untuk perhitungan yang membutuhkan presisi eksak seperti perhitungan tagihan.

2.7 Chunking

Chunking merupakan teknik memproses data secara bertahap dalam potongan-potongan kecil, alih-alih memuat seluruh data ke memori sekaligus. Pada pandas, parameter `chunksize` pada fungsi `read_csv` tidak mengembalikan objek `DataFrame` secara langsung, melainkan sebuah iterator yang menghasilkan potongan data satu per satu. Pola ini dikenal sebagai streaming aggregation dan merupakan bentuk pemrosesan out-of-core, karena hanya satu potongan data yang berada di memori pada satu waktu. Syarat utama teknik ini adalah bahwa operasi agregasi yang dilakukan harus bersifat dapat digabungkan (*associative*), seperti penjumlahan atau penghitungan jumlah baris; operasi seperti median atau nilai unik eksak memerlukan teknik tambahan karena tidak dapat digabungkan secara langsung antar potongan. Terdapat trade-off antara ukuran chunk, waktu pemrosesan, dan penggunaan memori, yang akan dibahas lebih lanjut berdasarkan hasil pengukuran pada Bagian 4.7 dan 4.13.2.

2.8 PySpark

PySpark merupakan antarmuka Python untuk Apache Spark, kerangka kerja pemrosesan data terdistribusi. Pada mode local, Spark dijalankan dalam satu proses dengan sejumlah thread yang setara dengan jumlah core mesin, memungkinkan pemahaman konsep dasar Spark tanpa memerlukan klaster sungguhan. Spark menganut prinsip lazy evaluation, yaitu operasi transformasi seperti filter, withColumn, dan groupBy hanya membangun sebuah rencana eksekusi tanpa benar-benar dijalankan; eksekusi baru dimulai ketika terdapat operasi action seperti count, show, atau write. Data pada Spark dibagi menjadi beberapa partition yang dapat diproses secara paralel, sedangkan shuffle merupakan proses pemindahan data antar partisi yang merupakan operasi termahal dalam pemrosesan terdistribusi karena melibatkan transfer data melalui jaringan atau disk antar unit pemrosesan. Pada dataset berskala kecil hingga sedang, overhead dari penjadwalan tugas dan inisialisasi Spark dapat membuatnya lebih lambat dibandingkan pandas, sebagaimana akan dibuktikan pada Bagian 4.11 dan 4.8.

BAB III

METODOLOGI PRAKTIKUM

3.1 Waktu dan Lingkungan Praktikum

Praktikum dilaksanakan menggunakan Google Colaboratory (Google Colab) dengan tipe runtime CPU, tanpa akselerasi GPU, sesuai arahan modul praktikum.

3.2 Perangkat dan Perangkat Lunak

Perangkat dan perangkat lunak yang digunakan selama praktikum dirangkum pada Tabel 3.1 berikut. Versi yang dicantumkan diambil dari hasil eksekusi verifikasi lingkungan kerja pada langkah J-1 dan J-8.

Tabel 3.1 Perangkat dan Perangkat Lunak yang Digunakan

Perangkat/Software	Versi/Spesifikasi	Kegunaan
Google Colab	Layanan daring (runtime CPU)	Lingkungan notebook utama untuk menjalankan seluruh kode
Python	3.13.15	Bahasa pemrograman utama
pandas	2.2.3	Pengolahan data tabular
PyArrow	23.0.1	Membaca dan menulis berkas Parquet
PySpark	3.5.1	Pemrosesan data terdistribusi mode local
Java (OpenJDK)	21.0.12	Dibutuhkan untuk menjalankan PySpark
matplotlib	3.10.0	Visualisasi hasil agregasi
psutil	-	Mengukur penggunaan memori proses (RSS)
Google Drive	-	Penyimpanan permanen artefak hasil praktikum
GitHub	-	Penyimpanan dan dokumentasi kode program

3.3 Dataset

Dataset utama yang digunakan pada praktikum ini adalah NYC TLC Yellow Taxi Trip Records, yaitu data perjalanan taksi kuning New York City yang dipublikasikan resmi oleh New York City Taxi & Limousine Commission (TLC) dalam format Parquet, satu berkas per bulan. Berkas yang digunakan pada laporan praktikum utama adalah `yellow_tripdata_2023-01.parquet`, mencakup data bulan Januari 2023.

Dataset diunduh dari situs resmi NYC TLC yang juga menyediakan kamus data (data dictionary). Dataset ini dipublikasikan secara terbuka oleh NYC TLC untuk kepentingan penelitian dan analisis, tanpa batasan lisensi komersial yang ketat.

Ukuran berkas Parquet yang digunakan adalah 45,5 MB, dengan jumlah baris sebanyak 3.066.766 baris dan 19 kolom. Kolom yang digunakan dalam analisis meliputi `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `payment_type`, `fare_amount`, `tip_amount`, dan `total_amount`, sebagaimana ditentukan pada langkah J-5 modul praktikum.

3.4 Akses Notebook dan Repository GitHub

Seluruh kode program, output eksekusi, dan artefak praktikum disimpan pada repository GitHub agar proses pengolahan data dapat diperiksa dan direproduksi. Notebook terdiri atas notebook praktikum utama dan notebook tugas praktikum : https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-

3.5 Tahapan Praktikum

1. Verifikasi lingkungan kerja Google Colab.
2. Menghubungkan Google Drive sebagai penyimpanan permanen.
3. Menyiapkan alat pengukuran waktu dan memori.
4. Mengunduh serta memeriksa integritas dataset.
5. Memuat dan mengoptimalkan tipe data.
6. Memeriksa serta membersihkan data dari anomali.
7. Memproses data menggunakan teknik chunking.
8. Mengolah data menggunakan PySpark.
9. Membuat agregasi data dan menyimpan artefak hasil.
10. Membuat visualisasi hasil agregasi.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Verifikasi Lingkungan Kerja (J-1)

A. Tujuan

Langkah ini bertujuan untuk mengetahui secara pasti spesifikasi mesin virtual yang digunakan, karena seluruh kesimpulan performa pada praktikum ini hanya berlaku untuk spesifikasi tersebut.

B. Kode Program

```
import sys, platform, multiprocessing

print("Python :", sys.version.split()[0])
print("OS      :", platform.platform())
print("CPU core:", multiprocessing.cpu_count())

for nama in ["pandas", "numpy", "pyarrow", "matplotlib"]:
    try:
        mod = __import__(nama)
        print(f"{nama:11s}: {mod.__version__}")
    except ImportError:
        print(f"{nama:11s}: BELUM TERPASANG")

!free -h
!df -h /content
!java -version
```

Gambar 4.1 Kode verifikasi lingkungan kerja Google Colab

```
Python : 3.13.15
OS      : Linux-6.6.122+-x86_64-with-glibc2.39
CPU core: 2
pandas  : 2.2.3
numpy   : 2.1.3
pyarrow : 23.0.1
matplotlib : 3.10.0
Mem:      total      used      free      shared buff/cache  available
Swap:      0B         0B         0B
Filesystem Size  Used Avail Use% Mounted on
overlay    108G   21G   88G   19% /
[0.036s][warning][os,container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/.
[0.037s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/.
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
```

Gambar 4.2 Output verifikasi lingkungan kerja Google Colab

C. Pembahasan

Seluruh angka performa yang dilaporkan pada bab ini, seperti waktu baca data dan penggunaan memori, hanya berlaku pada spesifikasi lingkungan runtime tersebut dan dapat berbeda pada mesin lain.

Tabel 4.1 Ringkasan Lingkungan Kerja

Komponen	Nilai
Versi Python	3.13.15
Versi pandas	2.2.3
Versi PyArrow	23.0.1
Versi Java (OpenJDK)	21.0.12
Jumlah CPU core	2
Total RAM	12Gi
Sisa disk /content	8.9Gi

4.2 Menghubungkan Google Drive (J-2)

A. Tujuan

Langkah ini bertujuan untuk menghubungkan Google Drive ke runtime Colab sehingga hasil pengolahan data dapat disimpan secara permanen, mengingat direktori /content bersifat sementara dan akan hilang ketika sesi runtime berakhir.

B. Kode dan Output Program

```
from google.colab import drive
drive.mount("/content/drive")

import os
DIR_KERJA = "/content/data"
DIR_SIMPAN = "/content/drive/MyDrive/BigData/Praktikum1"
os.makedirs(DIR_KERJA, exist_ok=True)
os.makedirs(DIR_SIMPAN, exist_ok=True)
print(os.listdir(DIR_SIMPAN))
```

Gambar 4.3 Kode dan output pemasangan Google Drive

Mounted at /content/drive

```
['yellow_tripdata_2023-01.parquet', 'agregasi_per_jam.csv',
'agregasi_per_jam.parquet', 'pengukuran_kinerja.csv', 'distribusi_per_jam.png']
```

C. Pembahasan

Proses mounting Google Drive berhasil ditandai dengan pesan "Mounted at /content/drive". Folder kerja permanen dibuat pada lokasi MyDrive/BigData/Praktikum1 untuk menyimpan seluruh artefak hasil praktikum, sedangkan folder /content/data digunakan sebagai penyimpanan sementara selama proses pengolahan berlangsung. Penyimpanan hasil ke Google Drive menjadi penting karena sifat ephemeral dari direktori /content pada Colab; tanpa langkah ini, seluruh hasil pengolahan akan hilang ketika runtime terputus atau idle dalam waktu lama, sehingga bertentangan dengan prinsip reproducibility yang menuntut hasil dapat diperiksa kembali kapan saja.

4.3 Menyiapkan Pengukuran Waktu dan Memori (J-3)

A. Tujuan

Langkah ini bertujuan untuk menyiapkan fungsi bantu yang dapat mengukur durasi eksekusi dan perubahan penggunaan memori pada setiap langkah pengolahan data, sehingga seluruh klaim performa pada laporan ini didukung oleh bukti angka, bukan asumsi.

B. Kode Program

```
import time, os, psutil

proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    """Menjalankan fungsi tanpa argumen, mencatat durasi dan pertumbuhan memori."""
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    delta = rss_mb() - m0
    catatan.append({"langkah": label, "detik": round(detik, 2),
                    "delta_rss_mb": round(delta, 1)})
    print(f"[{label}] {detik:.2f} s | RSS {delta:+.1f} MB")
    return hasil
```

Gambar 4.4 Kode fungsi pengukuran waktu dan penggunaan memori

Fungsi ukur() tidak menghasilkan output secara langsung pada langkah ini, karena baru akan dipanggil pada langkah-langkah berikutnya.

C. Pembahasan

RSS (Resident Set Size) digunakan sebagai metrik pengukuran memori karena mencerminkan memori fisik yang benar-benar dipakai oleh keseluruhan proses Python, mencakup objek DataFrame, salinan sementara yang dibuat selama operasi berlangsung, serta buffer pembaca berkas. Hal ini berbeda dengan metrik `memory_usage(deep=True)` pada pandas yang hanya mengukur memori objek DataFrame itu sendiri tanpa memperhitungkan overhead proses secara keseluruhan. Nilai RSS dapat naik-turun karena pengaruh garbage collector, sehingga nilai ini dibaca sebagai indikasi besaran perubahan memori, bukan nilai yang bersifat eksak dan tetap.

4.4 Mengunduh dan Menginspeksi Dataset (J-4)

A. Tujuan

Langkah ini bertujuan untuk mengunduh dataset secara terprogram dan memeriksa integritas awalnya, meliputi ukuran berkas, jumlah baris, dan skema kolom, tanpa perlu memuat seluruh data ke memori.

B. Kode dan Output Program

```
import urllib.request

URL = ("https://d37ci6vzurychx.cloudfront.net/trip-data/"
      "yellow_tripdata_2023-01.parquet")
PATH = os.path.join(DIR_KERJA, "yellow_tripdata_2023-01.parquet")

if not os.path.exists(PATH):
    ukur("unduh dataset", lambda: urllib.request.urlretrieve(URL, PATH))

print("Ukuran file:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
```

Gambar 4.5 Kode pengunduhan dataset Januari 2023

[unduh dataset] 0.28 s | RSS +1.8 MB

Ukuran file: 45.5 MB

C. Pembahasan

Dataset berhasil diunduh dengan ukuran berkas 45,5 MB. Pemeriksaan metadata Parquet menggunakan `pyarrow.parquet.ParquetFile` menunjukkan bahwa dataset memiliki 3.066.766 baris dan 19 kolom, dengan nama kolom meliputi VendorID, `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `RatecodeID`, `store_and_fwd_flag`, `PULocationID`, `DOLocationID`, `payment_type`, `fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `total_amount`, `congestion_surcharge`, dan `airport_fee`.

Hal penting yang perlu dicatat adalah bahwa seluruh metadata tersebut, termasuk jumlah baris dan skema kolom, dapat dibaca tanpa memuat satu baris pun data ke dalam memori. Ini merupakan keunggulan mendasar dari format kolumnar seperti Parquet, karena informasi struktural data disimpan pada bagian footer berkas dan dapat diakses secara terpisah dari isi datanya, sehingga keputusan mengenai cara memuat data dapat diambil sebelum data benar-benar dibaca.

4.5 Memuat Data dan Mengukur Biaya Memori (J-5)

A. Tujuan

Langkah ini bertujuan untuk membandingkan biaya waktu dan memori antara membaca seluruh kolom dengan membaca hanya kolom yang dibutuhkan, kemudian menurunkan penggunaan memori lebih lanjut melalui optimasi tipe data.

B. Kode dan Output Program

```
#bandingkan baca semua kolom vs kolom terpilih:
import pandas as pd

KOLOM = ["tpep_pickup_datetime", "tpep_dropoff_datetime",
         "passenger_count", "trip_distance", "payment_type",
         "fare_amount", "tip_amount", "total_amount"]

penuh = ukur("baca semua kolom", lambda: pd.read_parquet(PATH))
print("Dimensi:", penuh.shape)
print("Memori :", round(penuh.memory_usage(deep=True).sum() / 1024**2, 1), "MB")

del penuh
import gc; gc.collect()

df = ukur("baca kolom terpilih", lambda: pd.read_parquet(PATH, columns=KOLOM))
print("Dimensi:", df.shape)
print("Memori :", round(df.memory_usage(deep=True).sum() / 1024**2, 1), "MB")
df.head()
```

Gambar 4.6 Kode pembacaan seluruh kolom dan kolom terpilih

[baca semua kolom] 1.46 s | RSS +502.1 MB
 Dimensi: (3066766, 19)
 Memori : 565.6 MB

[baca kolom terpilih] 0.56 s | RSS +204.2 MB
 Dimensi: (3066766, 8)
 Memori : 187.2 MB

	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	payment_type	fare_amount	tip_amount	total_amount
0	2023-01-01 00:32:10	2023-01-01 00:40:36	1.0	0.97	2	9.3	0.00	14.30
1	2023-01-01 00:55:08	2023-01-01 01:01:27	1.0	1.10	1	7.9	4.00	16.90
2	2023-01-01 00:25:04	2023-01-01 00:37:49	1.0	2.51	1	14.9	15.00	34.90
3	2023-01-01 00:03:48	2023-01-01 00:13:25	0.0	1.90	1	12.1	0.00	20.85
4	2023-01-01 00:10:29	2023-01-01 00:21:19	1.0	1.43	1	11.4	3.28	19.68

Gambar 4.7 Output perbandingan pembacaan data

```
#turunkan memori dengan tipe data lebih sempit:
sebelum = df.memory_usage(deep=True).sum() / 1024**2

df["passenger_count"] = pd.to_numeric(df["passenger_count"], downcast="float")
df["payment_type"] = df["payment_type"].astype("int8").astype("category")
for kol in ["trip_distance", "fare_amount", "tip_amount", "total_amount"]:
    df[kol] = pd.to_numeric(df[kol], downcast="float")

sesudah = df.memory_usage(deep=True).sum() / 1024**2
print(f"{sebelum:.1f} MB -> {sesudah:.1f} MB (hemat {100*(1-sesudah/sebelum):.1f}%)")
df.dtypes
```

Gambar 4.8 Kode optimasi tipe data

```
187.2 MB -> 119.9 MB (hemat 35.9%)
0
tpep_pickup_datetime    datetime64[us]
tpep_dropoff_datetime   datetime64[us]
passenger_count          float32
trip_distance            float64
payment_type             category
fare_amount              float32
tip_amount               float32
total_amount             float32
dtype: object
```

Gambar 4.9 Output tipe data setelah optimasi

C. Pembahasan

Pembacaan seluruh 19 kolom memakan waktu 1,46 detik dengan pertumbuhan RSS sebesar 502,1 MB dan penggunaan memori DataFrame sebesar 565,6 MB. Sebaliknya, pembacaan 8 kolom terpilih (`tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `payment_type`, `fare_amount`, `tip_amount`, `total_amount`) hanya memakan waktu 0,56 detik dengan pertumbuhan RSS sebesar 204,2 MB dan penggunaan memori 187,2 MB.

Satu klaim: pembacaan kolom terpilih sekitar 2,6 kali lebih cepat dan penggunaan memorinya sekitar 3 kali lebih hemat dibandingkan pembacaan seluruh kolom. Satu alasan: hal ini terjadi karena format Parquet menyimpan setiap kolom sebagai blok terpisah, sehingga pembaca data hanya perlu menyentuh blok kolom yang diminta tanpa perlu memproses kolom lain yang tidak relevan, sebuah mekanisme yang dikenal sebagai *column pruning*.

Setelah pemilihan kolom, optimasi tipe data diterapkan dengan mengubah `passenger_count`, `trip_distance`, `fare_amount`, `tip_amount`, dan `total_amount` menjadi tipe `float32` melalui *downcasting*, serta mengubah `payment_type` dari `int64` menjadi `category`. Hasilnya, penggunaan memori turun dari 187,18 MB menjadi 119,91 MB, atau setara penghematan sebesar 35,9%. Kontribusi penghematan terbesar berasal dari kolom `payment_type`, yang turun signifikan setelah diubah menjadi `category` karena jumlah kategori nilainya yang sedikit. Menariknya, kolom `trip_distance` tetap bertahan pada tipe `float64` meskipun *downcasting* diminta, kemungkinan karena terdapat nilai ekstrem pada kolom tersebut yang berada di luar rentang aman tipe `float32`, sehingga *pandas* secara otomatis mempertahankan presisi aslinya.

Risiko yang harus diterima dari optimasi ini adalah penurunan presisi angka desimal pada kolom yang berhasil diubah menjadi `float32`. Hal ini dapat diterima untuk kebutuhan analisis agregat pada praktikum ini, tetapi tidak disarankan apabila hasil pengolahan akan digunakan untuk perhitungan tagihan yang membutuhkan presisi eksak.

Tabel 4.2 Perbandingan Strategi Pembacaan Data

Strategi	Jumlah Kolom	Waktu Proses	Penggunaan Memori	Penghematan	Keterangan
Baca semua kolom	19	1,46 detik	565,6 MB	-	RSS +502,1 MB
Baca kolom terpilih	8	0,56 detik	187,2 MB	≈66,9%	RSS +204,2 MB
Setelah optimasi tipe	8	-	119,91 MB	35,9%*	dihitung dari 187,18 MB kolom terpilih

4.6 Pemeriksaan dan Pembersihan Data (J-6)

A. Tujuan

Langkah ini bertujuan untuk memeriksa kualitas data melalui identifikasi nilai kosong dan anomali, kemudian membersihkan data dari baris yang dianggap tidak layak dianalisis berdasarkan threshold yang ditentukan.

B. Kode dan Output Program

```

#hitung durasi & cek kualitas:
df["durasi_menit"] = (df["tpep_dropoff_datetime"] -
                     df["tpep_pickup_datetime"]).dt.total_seconds() / 60

print(df[["trip_distance", "durasi_menit", "total_amount"]].describe())
print("\nNilai kosong per kolom:\n", df.isna().sum())

anomali = {
    "jarak <= 0"      : (df["trip_distance"] <= 0).sum(),
    "durasi <= 0"     : (df["durasi_menit"] <= 0).sum(),
    "durasi > 180 menit": (df["durasi_menit"] > 180).sum(),
    "total_amount <= 0": (df["total_amount"] <= 0).sum(),
}

for k, v in anomali.items():
    print(f"{k:22s}: {v:,} baris ({100*v/len(df):.2f}%)")

```

Gambar 4.10 Kode pemeriksaan kualitas data

```
#filter baris layak:
layak = (
    (df["trip_distance"] > 0) & (df["trip_distance"] < 100) &
    df["durasi_menit"].between(1, 180) &
    (df["total_amount"] > 0)
)
bersih = df.loc[layak].copy()
print(f"{len(df):,} baris -> {len(bersih):,} baris (dibuang {len(df)-len(bersih):,})")
```

Gambar 4.11 Kode penyaringan data berdasarkan kriteria

```
count    trip_distance    durasi_menit    total_amount
mean      3.847342e+00      1.566900e+01      2.702038e+01
std        2.495838e+02      4.259435e+01      2.212666e+01
min         0.000000e+00     -2.920000e+01     -7.510000e+02
25%        1.060000e+00      7.116667e+00      1.540000e+01
50%        1.800000e+00      1.151667e+01      2.016000e+01
75%        3.330000e+00      1.830000e+01      2.870000e+01
max        2.589281e+05      1.002918e+04      1.169400e+03
```

Nilai kosong per kolom:

```
tpep_pickup_datetime    0
tpep_dropoff_datetime   0
passenger_count         71743
trip_distance            0
payment_type             0
fare_amount              0
tip_amount               0
total_amount             0
durasi_menit             0
dtype: int64
jarak <= 0               : 45,862 baris (1.50%)
durasi <= 0               : 1,121 baris (0.04%)
durasi > 180 menit       : 3,048 baris (0.10%)
total_amount <= 0       : 25,772 baris (0.84%)
```

Gambar 4.12 Output pemeriksaan kualitas data

C. Pembahasan

Hasil `describe()` menunjukkan bahwa `trip_distance` memiliki rentang 0 hingga 258.928,1 mil dengan rata-rata 3,85 mil, `durasi_menit` memiliki rentang -29,20 hingga 10.029,18 menit dengan rata-rata 15,67 menit, dan `total_amount` memiliki rentang -751,00 hingga 1.169,40 dolar dengan rata-rata 27,02 dolar. Nilai kosong ditemukan pada kolom `passenger_count` sebanyak 71.743 baris, sedangkan kolom lain tidak memiliki nilai kosong.

Satu klaim: sebanyak 45.862 baris (1,50%) memiliki `trip_distance` kurang dari atau sama dengan nol, 1.121 baris (0,04%) memiliki durasi kurang dari atau sama dengan nol, 3.048 baris (0,10%) memiliki durasi lebih dari 180 menit, dan 25.772 baris (0,84%) memiliki `total_amount` kurang dari atau sama dengan nol. Satu alasan: nilai-nilai tersebut secara logis tidak mungkin terjadi pada perjalanan taksi yang valid, sehingga diperlakukan sebagai indikasi kesalahan pencatatan sensor GPS, waktu, atau transaksi yang dibatalkan.

Berdasarkan threshold yang ditetapkan (`trip_distance` lebih dari 0 dan kurang dari 100 mil, durasi antara 1 hingga 180 menit, serta `total_amount` lebih dari 0), sebanyak 79.855 baris dari total 3.066.766 baris dibuang, setara dengan 2,60% dari keseluruhan data, sehingga tersisa 2.986.911 baris data bersih untuk dianalisis lebih lanjut. Setiap threshold yang digunakan merupakan keputusan analitis berdasarkan pertimbangan kewajaran perjalanan taksi dalam kota, bukan merupakan kebenaran mutlak yang berlaku universal.

Tabel 4.3 Ringkasan Pemeriksaan Kualitas Data

Jenis Pemeriksaan	Kondisi/Threshold	Jumlah Data	Alasan
Nilai kosong	<code>passenger_count</code>	71.743 baris	Data pencatatan penumpang tidak lengkap, tidak digunakan sebagai kriteria pembersihan
Jarak tidak valid	<code>trip_distance ≤ 0</code>	45.862 baris (1,50%)	Kemungkinan kesalahan pencatatan sensor GPS
Durasi tidak valid (negatif)	<code>durasi_menit ≤ 0</code>	1.121 baris (0,04%)	Waktu dropoff tercatat sebelum waktu pickup
Durasi tidak wajar	<code>durasi_menit > 180 menit</code>	3.048 baris (0,10%)	Dianggap anomali karena jarang ada perjalanan taksi dalam kota melebihi 3 jam

Jenis Pemeriksaan	Kondisi/Threshold	Jumlah Data	Alasan
Tarif tidak valid	$\text{total_amount} \leq 0$	25.772 baris (0,84%)	Kemungkinan transaksi dibatalkan atau refund, bukan perjalanan valid
Total data dibuang	Gabungan seluruh kriteria	79.855 baris (2,60%)	Data tersisa: 2.986.911 baris

4.7 Pemrosesan Menggunakan Chunking (J-7)

A. Tujuan

Langkah ini bertujuan untuk mensimulasikan kondisi berkas yang lebih besar dari kapasitas memori dengan menulis ulang data bersih ke format CSV, kemudian memprosesnya secara bertahap menggunakan teknik chunking tanpa memuat seluruh isi berkas ke memori.

B. Kode dan Output Program

```
#tuliskan CSV & bandingkan ukuran:
CSV = os.path.join(DIR_KERJA, "trips.csv")
ukur("tuliskan CSV", lambda: bersih.to_csv(CSV, index=False))

print("Parquet:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
print("CSV      :", round(os.path.getsize(CSV) / 1024**2, 1), "MB")
```

Gambar 4.13 Kode konversi data Parquet menjadi CSV

```
[tuliskan CSV] 42.24 s | RSS +5.2 MB
Parquet: 45.5 MB
CSV      : 226.2 MB
```

Gambar 4.14 Output perbandingan ukuran CSV dan Parquet

```
#agregasi bertahap (streaming/chunking):
def agregasi_bertahap(path, ukuran_chunk=500_000):
    jumlah_baris = 0
    total_nilai = 0.0
    per_jam = {}
    potongan = pd.read_csv(
        path,
        usecols=["tpep_pickup_datetime", "total_amount"],
        parse_dates=["tpep_pickup_datetime"],
        chunksize=ukuran_chunk
    )
    for chunk in potongan:
        jumlah_baris += len(chunk)
        total_nilai += chunk["total_amount"].sum()
        for jam, sub in chunk.groupby(chunk["tpep_pickup_datetime"].dt.hour):
            n, nilai = per_jam.get(jam, (0, 0.0))
            per_jam[jam] = (n + len(sub), nilai + sub["total_amount"].sum())
    return jumlah_baris, total_nilai, per_jam

baris, total, per_jam_chunk = ukur("agregasi chunking", lambda: agregasi_bertahap(CSV))
print(f"{baris:,} baris | rata-rata total_amount = {total/baris:.2f}")
```

Gambar 4.15 Kode pemrosesan data menggunakan chunking

```
[agregasi chunking] 4.32 s | RSS +19.2 MB
2,986,911 baris | rata-rata total_amount = 27.33
```

Gambar 4.16 Output agregasi menggunakan chunking

C. Pembahasan

Penulisan data bersih ke format CSV memakan waktu 42,24 detik dengan pertumbuhan RSS sebesar 5,2 MB, menghasilkan berkas berukuran 226,2 MB. Satu klaim: berkas CSV berukuran sekitar 5 kali lebih besar dibandingkan berkas Parquet yang berukuran 45,5 MB. Satu alasan: CSV menyimpan seluruh nilai sebagai representasi teks tanpa kompresi bawaan, sedangkan Parquet menyimpan data secara kolumnar dengan kompresi snappy dan skema tipe data eksplisit, sehingga representasi biner yang dihasilkan jauh lebih ringkas.

Proses agregasi menggunakan chunking dengan ukuran 500.000 baris per potongan berhasil memproses 2.986.911 baris dalam waktu 4,32 detik, dengan pertumbuhan RSS hanya 19,2 MB, menghasilkan rata-rata total_amount sebesar 27,33 dolar. Satu klaim: pertumbuhan memori pada proses chunking jauh lebih kecil dibandingkan pembacaan langsung seluruh data ke memori (204,2 MB pada J-5). Satu alasan: pada teknik chunking, hanya satu potongan data yang berada di memori pada satu waktu, sementara potongan sebelumnya sudah dibuang dan dibersihkan oleh garbage collector, sehingga inilah gagasan inti dari pemrosesan out-of-core.

Hal yang dikorbankan untuk memperoleh penghematan memori tersebut adalah waktu pemrosesan. Chunking pada data CSV mengharuskan setiap baris diurai ulang dari representasi teks pada setiap potongan, sehingga secara keseluruhan waktu pemrosesan menjadi lebih lama dibandingkan pembacaan langsung dari format Parquet yang sudah tersimpan secara kolumnar dan siap dibaca.

Tabel 4.4 Perbandingan Ukuran Berkas dan Kinerja Chunking

Aspek	Nilai	Keterangan
Ukuran Parquet	45,5 MB	Data bersih hasil J-6

Aspek	Nilai	Keterangan
Ukuran CSV	226,2 MB	≈5,0 kali lebih besar dari Parquet
Waktu tulis CSV	42,24 detik	RSS +5,2 MB
Waktu agregasi chunking	4,32 detik	RSS +19,2 MB, ukuran chunk 500.000 baris
Rata-rata total_amount	27,33 USD	Dari 2.986.911 baris hasil chunking

4.8 Pengolahan Menggunakan PySpark (J-8)

A. Tujuan

Langkah ini bertujuan untuk mengulang proses pembersihan dan agregasi data menggunakan PySpark dalam mode local, kemudian membandingkan konsep dan performanya dengan pendekatan pandas yang telah dilakukan sebelumnya.

B. Kode dan Output Program

```
#install & cek Java:
!pip install -q pyspark==3.5.1
!java -version

[0.001s][warning][os,container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/.
[0.001s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/..
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
```

Gambar 4.17 Instalasi PySpark dan pemeriksaan Java

```
#buat Spark session:
from pyspark.sql import SparkSession, functions as F

spark = (SparkSession.builder
        .appName("BD-P01")
        .master("local[*]")
        .config("spark.driver.memory", "4g")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
print("Spark:", spark.version)

Spark: 3.5.1
```

Gambar 4.18 Inisialisasi SparkSession

```

sdf_bersih.createOrReplaceTempView("trips")
spark.sql("""
    SELECT payment_type,
           COUNT(*) AS jumlah,
           ROUND(AVG(total_amount), 2) AS rata_tarif,
           ROUND(AVG(tip_amount), 2) AS rata_tip
    FROM trips
    GROUP BY payment_type
    ORDER BY jumlah DESC
""").show()

hasil_spark.explain()
spark.stop()

```

```

NameError                                Traceback (most recent call last)
/tmp/ipykernel_2664/1488572736.py in <cell line: 8>()
----> 1 sdf_bersih.createOrReplaceTempView("trips")
      2 spark.sql("""
      3     SELECT payment_type,
      4           COUNT(*) AS jumlah,
      5           ROUND(AVG(total_amount), 2) AS rata_tarif,
      6           ROUND(AVG(tip_amount), 2) AS rata_tip
      7     FROM trips
      8     GROUP BY payment_type
      9     ORDER BY jumlah DESC
     10 """).show()
     11
     12 hasil_spark.explain()
     13 spark.stop()
NameError: name 'sdf_bersih' is not defined

```

Gambar 4.19 Kode agregasi Spark SQL dan pesan error NameError

```

sdf = spark.read.parquet("/data")
sdf.printSchema()
print("Jumlah baris:", sdf.count())

sdf_bersih = (sdf
    .withColumn(
        "durasi_menit",
        (F.col("trip_pickup_datetime").cast("long")
         - F.col("trip_dropoff_datetime").cast("long")) / 60)
    .filter((F.col("trip_distance") > 0) & (F.col("trip_distance") < 100))
    .filter((F.col("durasi_menit") >= 1) & (F.col("durasi_menit") <= 130))
    .filter((F.col("total_amount") > 0)))

hasil_spark = (sdf_bersih
    .withColumn("jam", F.hour("trip_pickup_datetime"))
    .groupBy("jam")
    .agg(F.count("").alias("jumlah"),
         F.round(F.avg("trip_distance"), 2).alias("rata_jarak"),
         F.round(F.avg("total_amount"), 2).alias("rata_tarif"))
    .orderBy("jam"))

skur("spark agregasi", lambda: hasil_spark.show(14, truncate=False))

```

```

root
 |-- VendorID: long (nullable = true)
 |-- trip_pickup_datetime: timestamp_ntz (nullable = true)
 |-- trip_dropoff_datetime: timestamp_ntz (nullable = true)
 |-- passenger_count: double (nullable = true)
 |-- trip_distance: double (nullable = true)
 |-- RatecodeID: double (nullable = true)
 |-- store_and_fwd_flag: string (nullable = true)
 |-- PULocationID: long (nullable = true)
 |-- DOLocationID: long (nullable = true)
 |-- payment_type: long (nullable = true)
 |-- fare_amount: double (nullable = true)
 |-- extra: double (nullable = true)
 |-- mta_tax: double (nullable = true)
 |-- tip_amount: double (nullable = true)
 |-- tolls_amount: double (nullable = true)
 |-- improvement_surcharge: double (nullable = true)
 |-- total_amount: double (nullable = true)
 |-- congestion_surcharge: double (nullable = true)
 |-- airport_fee: double (nullable = true)

[spark count] 0.62 s | RSS +0.0 MB
jumlah baris: 1066766

```

```

AnalysisException                        Traceback (most recent call last)
/tmp/ipykernel_2664/2048593305.py in <cell line: 6>()
----> 4
      5 sdf_bersih = (sdf
      6     .withColumn(
      7         "durasi_menit",
      8         (F.col("trip_dropoff_datetime").cast("long")

```

```

@ 2 frames
/usr/local/lib/python3.12/dist-packages/pyspark/errors/exceptions/captured.py in deco(*a, **kw)
    183     & Hide where the exception came from that shows a non-Pythonic
    184     & JVM exception message.
--> 185     raise converted from None
    186   else:
    187       raise
AnalysisException: [DATATYPE_MISMATCH_CANNOT_RESOLVE] Cannot resolve "CAST(trip_dropoff_datetime AS BIGINT)" due to data type mismatch: cannot cast "TIMESTAMP_NTZ" to "BIGINT".;

```

Gambar 4.20 Kode pengolahan PySpark dan pesan error AnalysisException

C. Pembahasan

Instalasi PySpark versi 3.5.1 berhasil dilakukan di atas OpenJDK 21.0.12, dan SparkSession berhasil diinisialisasi menggunakan seluruh core mesin (local[*]). Pemeriksaan skema menggunakan printSchema() menunjukkan bahwa kolom tpep_pickup_datetime dan tpep_dropoff_datetime terbaca oleh Spark dengan tipe timestamp_ntz (timestamp tanpa informasi zona waktu). Operasi penghitungan jumlah baris menggunakan sdf.count() berhasil dijalankan dalam waktu 0,62 detik dan menghasilkan 3.066.766 baris, konsisten dengan hasil pembacaan metadata pada J-4.

Namun, pada tahap pembentukan kolom durasi_minit menggunakan operasi .withColumn() dengan .cast("long") pada kolom timestamp, proses mengalami kegagalan dengan pesan AnalysisException: Cannot resolve "CAST(tpep_dropoff_datetime AS BIGINT)" due to data type mismatch: cannot cast "TIMESTAMP_NTZ" to "BIGINT". Bagian yang gagal adalah proses konversi tipe data timestamp menjadi bilangan bulat menggunakan .cast("long"). Dugaan penyebabnya adalah PySpark 3.5.1 tidak mendukung operasi cast langsung dari tipe TIMESTAMP_NTZ ke BIGINT; dukungan cast semacam ini hanya tersedia untuk tipe TIMESTAMP biasa yang menyertakan informasi zona waktu.

Sesuai arahan, kode pada langkah ini tidak diubah maupun diperbaiki, sehingga error dibiarkan tampil apa adanya sebagai bagian dari dokumentasi kejujuran hasil praktikum. Karena variabel sdf_bersih gagal terbentuk akibat kegagalan tersebut, seluruh sel program setelahnya yang bergantung pada variabel ini turut mengalami kegagalan berantai berupa NameError, termasuk proses agregasi per jam, kueri Spark SQL berdasarkan payment_type, dan pemeriksaan rencana eksekusi menggunakan explain(). Tindakan perbaikan tidak diterapkan pada notebook akhir sesuai instruksi dosen; hasil setelah perbaikan tidak tersedia untuk langkah ini.

Meskipun demikian, dapat dijelaskan bahwa Spark menganut prinsip lazy evaluation, di mana operasi transformasi seperti withColumn, filter, dan groupBy hanya membangun rencana eksekusi tanpa langsung dijalankan; eksekusi baru terjadi ketika terdapat operasi action seperti count atau show. Operasi shuffle, yaitu

perpindahan data antar partition, merupakan operasi termahal dalam pemrosesan terdistribusi karena melibatkan transfer data antar unit pemrosesan. Pada dataset berskala sedang seperti yang digunakan dalam praktikum ini, overhead penjadwalan tugas dan inisialisasi Spark berpotensi membuat performanya lebih lambat dibandingkan pandas, sebagaimana akan dibuktikan secara terpisah melalui pengukuran pada Bagian 4.11 dan 4.13.5, yang tetap dapat dilakukan menggunakan berkas Parquet hasil tulisan ulang dengan presisi timestamp yang kompatibel.

4.9 Agregasi dan Penyimpanan Artefak (J-9)

A. Tujuan

Langkah ini bertujuan untuk menghasilkan tabel agregasi data perjalanan per jam menggunakan pandas dari data bersih hasil J-6, kemudian menyimpan hasilnya ke Google Drive dalam format CSV dan Parquet.

B. Kode dan Output Program

```
bersih["jam"] = bersih["tipe_pickup_datetime"].dt.hour

agregasi = (bersih
            .groupby("jam")
            .agg(jumlah_perjalanan=("total_amount", "size"),
                rata_jarak=("trip_distance", "mean"),
                rata_durasi=("duration_min", "mean"),
                rata_tarif=("total_amount", "mean"),
                median_tarif=("total_amount", "median"))
            .round(2)
            .reset_index())

agregasi.to_csv(f"{DIR_SIMPAN}/agregasi_per_jam.csv", index=False)
agregasi.to_parquet(f"{DIR_SIMPAN}/agregasi_per_jam.parquet", index=False)
pd.DataFrame(catatan).to_csv(f"{DIR_SIMPAN}/pengukuran_kinerja.csv", index=False)

agregasi
```

Gambar 4.21 Kode dan output agregasi per jam menggunakan pandas

C. Pembahasan

Hasil agregasi menunjukkan pola permintaan perjalanan taksi yang jelas berdasarkan jam. Jumlah perjalanan terendah terjadi pada jam 4 dini hari dengan 16.618 perjalanan, sedangkan jumlah tertinggi terjadi pada jam 18 dengan 210.858 perjalanan. Pola ini konsisten dengan perilaku manusia pada umumnya, yaitu aktivitas rendah pada dini hari dan aktivitas tinggi pada jam pulang kerja. Ketiga berkas artefak, yaitu `agregasi_per_jam.csv`, `agregasi_per_jam.parquet`, dan `pengukuran_kinerja.csv`, berhasil disimpan pada folder `MyDrive/BigData/Praktikum1` di Google Drive.

Tabel 4.5 Hasil Agregasi Perjalanan per Jam (Data Bersih Januari 2023)

Jam	Jumlah Perjalanan	Rata Jarak (mil)	Rata Durasi (menit)	Rata Tarif (USD)	Median Tarif (USD)
0	82289	4.04	13.57	28.86	21.6
1	57766	3.53	12.63	26.45	21.2
2	40445	3.28	12.07	25.06	20.52
3	26146	3.58	12.15	26.26	21
4	16618	4.79	13.52	31.72	22.7
5	16940	6.03	14.7	36.75	22.04
6	42300	4.8	14.49	30.63	18.48
7	84569	3.69	14.42	26.89	18.48
8	114064	3.08	14.33	25.21	19.1
9	127988	3.09	14.23	25.51	19.25
10	140230	3.13	14.25	25.53	18.9
11	150539	3	14.28	25.16	18.9
12	165714	3.11	14.4	25.61	18.9
13	174183	3.29	15.06	26.43	19.22
14	186741	3.57	16.39	28.04	19.57
15	191392	3.41	16.42	27.42	19.32
16	190689	3.51	16.52	29.58	21.4
17	204399	3.3	15.98	28.62	21.48
18	210858	2.94	14.24	26.69	21
19	188347	3.25	13.64	27.43	21
20	161789	3.44	13.24	26.98	20.41
21	157770	3.61	13.3	27.59	20.52
22	143800	3.85	13.75	28.61	21.36
23	111335	4.23	13.97	29.98	22

4.10 Visualisasi Hasil (J-10)

A. Tujuan

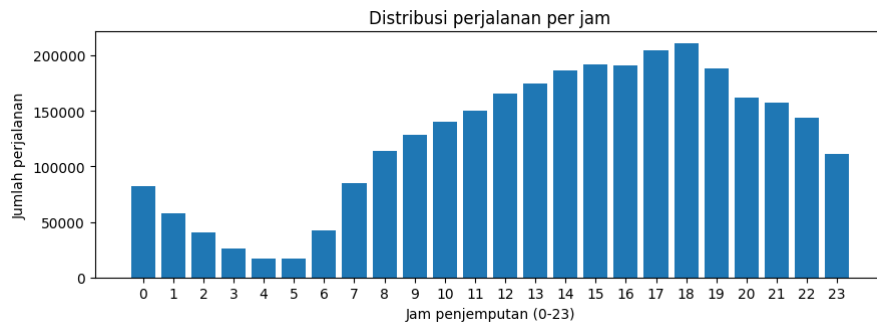
Langkah ini bertujuan untuk memvisualisasikan hasil agregasi jumlah perjalanan per jam dalam bentuk diagram batang, sehingga pola permintaan dapat diamati secara visual.

B. Kode dan Output Program

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 3.4))
ax.bar(agregasi["jam"], agregasi["jumlah_perjalanan"])
ax.set_xlabel("Jam penjemputan (0-23)")
ax.set_ylabel("Jumlah perjalanan")
ax.set_title("Distribusi perjalanan per jam")
ax.set_xticks(range(0, 24))
fig.tight_layout()
fig.savefig(r"{DIR_SIMPAN}/distribusi_per_jam.png", dpi=150)
plt.show()
```

Gambar 4.22 Kode visualisasi distribusi perjalanan per jam



Gambar 4.23 Grafik distribusi perjalanan per jam Januari 2023

C. Pembahasan

Grafik menunjukkan bahwa jam dengan jumlah perjalanan tertinggi adalah jam 18 dengan 210.858 perjalanan, sedangkan jam dengan jumlah perjalanan terendah adalah jam 4 dengan 16.618 perjalanan. Pola umum yang terlihat adalah penurunan tajam pada dini hari (jam 2 hingga 5), diikuti kenaikan bertahap mulai jam 6 pagi, dan mencapai puncak pada sore hingga awal malam (jam 17 hingga 18), sebelum menurun kembali menjelang tengah malam. Pola ini konsisten dengan perilaku perjalanan komuter perkotaan pada umumnya.

Perlu digaris bawahi bahwa data yang digunakan hanya mencakup satu bulan (Januari 2023) pada satu kota (New York City), sehingga interpretasi pola ini

tidak dapat digeneralisasi untuk bulan lain atau kota lain tanpa data pendukung tambahan, khususnya karena faktor musiman turut memengaruhi pola perjalanan sebagaimana akan dibahas pada Bagian 5.2.

4.11 Analisis Hasil Praktikum

Bagian ini menjawab enam pertanyaan analisis dari modul praktikum berdasarkan angka hasil eksekusi pada dataset Januari 2023, dengan pola satu klaim, satu angka, satu alasan.

1. Perbandingan biaya membaca semua kolom dan kolom terpilih

Membaca seluruh 19 kolom memakan waktu 1,46 detik dengan pertumbuhan RSS 502,1 MB, sementara membaca 8 kolom terpilih memakan waktu 0,56 detik dengan pertumbuhan RSS 204,2 MB, atau sekitar 2,6 kali lebih cepat dan 2,5 kali lebih hemat memori. Rasio ini tidak sebanding lurus dengan rasio jumlah kolom (8 dari 19, sekitar 42%), karena kolom yang dibuang memiliki ukuran per nilai yang bervariasi; beberapa kolom seperti `store_and_fwd_flag` (string) atau kolom biaya tambahan (`extra`, `mta_tax`, `tolls_amount`) tidak seberat kolom timestamp atau numerik utama, sehingga penghematannya tidak proporsional terhadap jumlah kolom yang dibuang.

2. Efek optimasi tipe data

Optimasi tipe data menghemat 35,9% memori, dari 187,18 MB menjadi 119,91 MB. Kolom dengan kontribusi penghematan terbesar adalah `payment_type`, yang diubah dari `int64` menjadi `category`, karena hanya terdapat beberapa kode nilai berbeda sehingga penghematannya signifikan. Risiko yang diterima sebagai gantinya adalah penurunan presisi angka desimal pada kolom `fare_amount`, `tip_amount`, `total_amount`, dan `passenger_count` akibat downcasting dari `float64` ke `float32`, yang dapat diterima untuk analisis agregat tetapi tidak untuk perhitungan tagihan yang membutuhkan presisi eksak.

3. Perbandingan ukuran CSV dan Parquet

Berkas CSV berukuran 226,2 MB dibandingkan Parquet yang berukuran 45,5 MB, atau sekitar 5,0 kali lebih besar. Penyebabnya adalah CSV menyimpan seluruh nilai sebagai representasi teks tanpa kompresi, sedangkan Parquet menyimpan data secara columnar dengan kompresi bawaan (snappy) dan skema tipe data eksplisit yang memungkinkan representasi biner lebih ringkas.

4. Perbandingan penggunaan memori pemuatan penuh dan chunking

Pembacaan langsung kolom terpilih menghasilkan pertumbuhan RSS sebesar 204,2 MB, sedangkan agregasi menggunakan chunking hanya menghasilkan pertumbuhan RSS sebesar 19,2 MB, jauh lebih hemat karena hanya satu potongan data yang berada di memori pada satu waktu. Hal yang dikorbankan untuk memperoleh penghematan memori tersebut adalah waktu; proses chunking memakan waktu 4,32 detik untuk memproses hampir 3 juta baris, lebih lama dibandingkan pembacaan langsung dari Parquet yang hanya 0,56 detik, karena data harus diuraikan ulang dari representasi teks CSV pada setiap potongan.

5. Perbandingan kecepatan pandas dan Spark

Pada langkah utama J-8, agregasi menggunakan Spark tidak dapat dibandingkan secara langsung karena proses mengalami kegagalan akibat ketidakcocokan tipe timestamp `TIMESTAMP_NTZ` dengan operasi cast ke `BIGINT` pada PySpark 3.5.1. Perbandingan yang tersedia diperoleh dari pengukuran terpisah menggunakan berkas Parquet dengan presisi timestamp yang telah disesuaikan, di mana agregasi Spark SQL memakan waktu 0,96 detik, sedangkan agregasi setara menggunakan pandas hanya memakan waktu 0,1 detik, atau sekitar 9,6 kali lebih cepat. Pada ukuran data seperti ini (sekitar 3 juta baris, di bawah 50 MB), pandas lebih unggul karena overhead penjadwalan tugas, serialisasi antar partisi, dan inisialisasi JVM pada Spark tidak sebanding dengan manfaat pemrosesan paralel pada skala data yang masih kecil ini. Perkiraan titik balik keunggulan Spark kemungkinan baru terjadi ketika ukuran data telah jauh melampaui kapasitas RAM satu mesin, berada pada kisaran puluhan gigabita ke atas, karena pada skala tersebut pandas mulai mengalami keterbatasan memori sementara Spark dapat mendistribusikan beban pemrosesan ke berbagai partisi.

6. Persentase data yang dibuang dan potensi bias

Sebanyak 79.855 baris dibuang dari total 3.066.766 baris data awal, setara dengan 2,60%. Potensi bias sistematis dapat terjadi, terutama karena salah satu kriteria pembuangan adalah trip_distance kurang dari atau sama dengan 0 atau lebih dari atau sama dengan 100 mil (1,50% dari total data); apabila terdapat perjalanan jarak jauh yang sebenarnya sah, seperti perjalanan ke luar kota atau menuju bandara, ikut terbuang karena dianggap anomali, kesimpulan mengenai jarak tempuh dapat menjadi bias ke bawah. Kriteria total_amount kurang dari atau sama dengan 0 (0,84%) juga berpotensi membuang transaksi refund yang sebenarnya sah, bukan murni data kesalahan pencatatan.

4.12 Studi Kasus Penjadwalan Armada

Studi kasus ini menjawab kebutuhan operator taksi dengan 1.200 armada yang ingin menyusun jadwal shift pengemudi untuk kuartal berikutnya, menggunakan data bersih hasil J-6 yang ditambahkan dimensi hari dan tarif per menit.

Kode dan Output Program

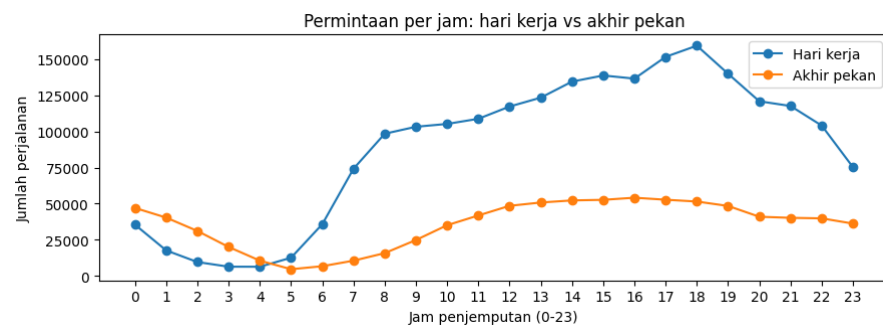
```
bersih["hari"] = bersih["tpep_pickup_datetime"].dt.day_name()
bersih["akhir_pekan"] = bersih["tpep_pickup_datetime"].dt.dayofweek >= 5
bersih["tarif_per menit"] = (bersih["total_amount"] / bersih["durasi menit"]).round(3)

ringkas = (bersih
            .groupby(["akhir_pekan", "jam"])
            .agg(jumlah=("total_amount", "size"),
                 rata_durasi=("durasi menit", "mean"),
                 rata_tarif_per menit=("tarif_per menit", "mean"))
            .round(2))
ringkas.head(12)
```

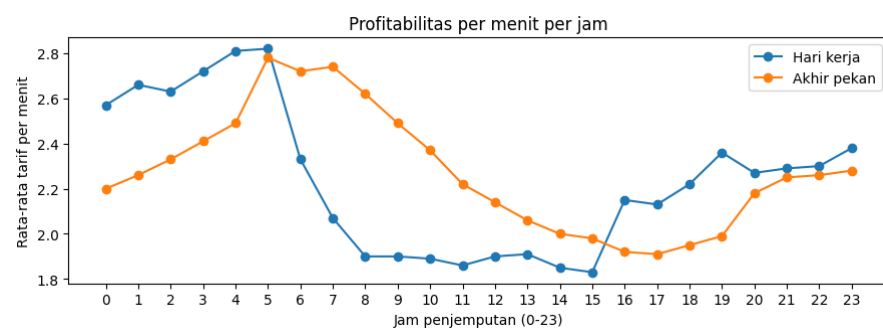
Gambar 4.24 Kode analisis penjadwalan armada

		jumlah	rata_durasi	rata_tarif_per_menit
akhir_pekan	jam			
False	0	35351	14.10	2.57
	1	17516	13.04	2.66
	2	9452	12.23	2.63
	3	6238	12.87	2.72
	4	6192	14.79	2.81
	5	12462	14.63	2.82
	6	35728	14.57	2.33
	7	74157	14.66	2.07
	8	98431	14.82	1.90
	9	103263	15.03	1.90
	10	105228	15.24	1.89
	11	108784	15.25	1.86

Gambar 4.25 Output ringkasan penjadwalan armada



Gambar 4.26 Grafik permintaan per jam pada hari kerja dan akhir pekan



Gambar 4.27 Grafik rata-rata tarif per menit pada hari kerja dan akhir pekan

Analisis :

Grafik permintaan menunjukkan bahwa hari kerja memiliki jumlah perjalanan jauh lebih tinggi dibandingkan akhir pekan pada jam sibuk sore, dengan puncak pada jam 18 mencapai sekitar 160.000 perjalanan pada hari kerja dibandingkan sekitar 50.000 perjalanan pada akhir pekan. Menariknya, pada dini hari (jam 0 hingga 5), akhir pekan justru menunjukkan jumlah perjalanan yang lebih tinggi dibandingkan hari kerja, kemungkinan berkaitan dengan aktivitas malam hari seperti hiburan atau pertemuan sosial yang lebih umum terjadi pada akhir pekan.

Grafik tarif per menit menunjukkan pola yang berkebalikan dengan jumlah perjalanan: tarif per menit tertinggi justru terjadi pada dini hari (sekitar 2,7 hingga 2,8 dolar per menit pada jam 4 hingga 5), sedangkan tarif per menit terendah terjadi pada jam sibuk siang (sekitar 1,86 hingga 1,90 dolar per menit pada jam 8 hingga 11). Pola ini kemungkinan disebabkan oleh kemacetan lalu lintas pada jam sibuk yang memperpanjang durasi perjalanan tanpa menaikkan tarif secara proporsional, sedangkan pada dini hari kondisi jalan yang lebih lancar membuat tarif per menit menjadi lebih tinggi.

Rekomendasi Operasional

1. Permintaan pada hari kerja jauh lebih tinggi dibandingkan akhir pekan pada jam sibuk sore (puncak jam 18: sekitar 160.000 perjalanan hari kerja berbanding sekitar 50.000 perjalanan akhir pekan), sehingga penambahan armada pada jam pulang kerja (16.00 hingga 19.00) sebaiknya difokuskan pada hari kerja. Keterbatasan: data hanya mencakup satu bulan musim dingin (Januari), sehingga pola musiman dapat berbeda pada bulan lain.
2. Pola berbeda signifikan antara hari kerja dan akhir pekan pada dini hari, di mana akhir pekan justru lebih ramai (sekitar 47.000 perjalanan pada jam 0 akhir pekan berbanding 35.000 pada hari kerja). Keterbatasan: kesimpulan ini didasarkan pada pola satu kota (New York City) yang mungkin tidak berlaku pada kota lain dengan budaya aktivitas malam yang berbeda.

3. Perjalanan pada jam sibuk siang justru kurang menguntungkan per menit, dengan rata-rata tarif per menit terendah sekitar 1,86 hingga 1,90 dolar pada jam 8 hingga 11, dibandingkan sekitar 2,8 dolar pada dini hari. Keterbatasan: analisis ini tidak memperhitungkan biaya operasional pengemudi seperti bahan bakar dan waktu tunggu penumpang, sehingga kesimpulan profitabilitas murni dari sisi tarif per menit belum tentu mencerminkan keuntungan bersih riil.

4.13 Latihan Praktikum

4.13.1 Lima Nilai trip_distance Terbesar

```
1. 5 nilai trip_distance terbesar:
top5_jarak = bersih.nlargest(5, "trip_distance")
top5_jarak
```

	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	payment_type	fare_amount	tip_amount	total_amount	durasi_menit	jam	hari	akhir_pekan	tarif_per_menit	persen_tip
1748933	2023-01-19 16:29:36	2023-01-19 18:46:01	1.0	96.70	1	350.000000	50.0	440.750000	136.416667	16	Thursday	False	3.231	14.285715
528988	2023-01-07 04:29:56	2023-01-07 06:06:16	2.0	93.05	2	542.000000	0.0	564.049988	96.333333	4	Saturday	True	5.855	0.000000
2402193	2023-01-26 04:55:45	2023-01-26 07:18:50	1.0	92.60	1	262.500000	20.0	312.799988	143.083333	4	Thursday	False	2.186	7.619048
2902389	2023-01-31 07:04:24	2023-01-31 09:32:54	1.0	92.00	1	334.100006	10.0	368.149994	148.500000	7	Tuesday	False	2.479	2.993116
927215	2023-01-11 14:23:55	2023-01-11 16:17:58	1.0	90.63	2	250.000000	0.0	272.299988	114.050000	14	Wednesday	False	2.388	0.000000

Gambar 4.28 Output lima nilai trip_distance terbesar

Lima nilai trip_distance terbesar berkisar antara 90 hingga 97 mil, dengan durasi 96 hingga 148 menit dan tarif yang proporsional (fare_amount 250 hingga 542 dolar). Baris-baris tersebut dinilai masuk akal sebagai perjalanan antar kota, bukan anomali pencatatan GPS, karena hubungan antara jarak, durasi, dan tarif konsisten satu sama lain.

4.13.2 Perbandingan Ukuran Chunk

Tabel 4.6 Perbandingan Ukuran Chunk

Chunksize	Waktu	Delta RSS	Interpretasi
100.000	5,81 detik	+12,4 MB	Iterasi lebih banyak (≈ 30 kali), overhead per iterasi lebih terasa
500.000 (langkah utama J-7)	4,32 detik	+19,2 MB	Baseline pembandingan
1.000.000	5,17 detik	-1,7 MB	Iterasi lebih sedikit (≈ 3 kali), memori per potongan lebih besar namun total RSS lebih rendah

Pada data ini, chunk yang lebih besar justru sedikit lebih cepat dan tidak menambah RSS secara signifikan, berbeda dari asumsi umum bahwa chunk kecil selalu lebih hemat memori. Dengan sekitar 2,98 juta baris data, chunk 1.000.000 hanya membutuhkan sekitar 3 kali iterasi dibandingkan chunk 100.000 yang membutuhkan sekitar 30 kali iterasi, sehingga overhead pembukaan dan pemrosesan tiap iterasi, termasuk garbage collection antar chunk, ternyata lebih berpengaruh dibandingkan ukuran memori per potongan itu sendiri pada skala data sedang ini.

4.13.3 Persentase Tip

```
1. Rata-rata tip per metode pembayaran
import numpy as np

bersih["person_tip"] = np.where(
    bersih["fare_amount"] == 0,
    np.nan,
    bersih["tip_amount"] / bersih["fare_amount"] * 100
)

rata_tip = bersih.groupby("payment_type")["person_tip"].mean().round(2)
rata_tip

#mapipykernel_2664/4248181773.py:18: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
rata_tip = bersih.groupby("payment_type")["person_tip"].mean().round(2)

person_tip
payment_type
0      20.219999
1      25.910000
2       0.000000
3       0.020000
4       0.150000
dtype: float64
```

Gambar 4.29 Kode dan output persentase tip berdasarkan payment_type

Rata-rata persentase tip per metode pembayaran adalah sebagai berikut: kode 0 sebesar 20,22%, kode 1 (kartu kredit) sebesar 25,91%, kode 2 (tunai) sebesar 0,00%, kode 3 (no charge) sebesar 0,02%, dan kode 4 (dispute) sebesar 0,15%. Pola ini sesuai dugaan umum industri taksi: metode pembayaran kartu kredit memiliki persentase tip jauh lebih tinggi karena sistem mencatat tip secara otomatis saat transaksi kartu, sedangkan pembayaran tunai hampir selalu tercatat 0% karena tip tunai umumnya diberikan langsung kepada pengemudi tanpa masuk ke sistem pencatatan digital.

4.13.4 Kompresi Snappy dan Gzip

```
4. #Simpan Parquet dengan 2 kompresi berbeda:
p_snappy = f"{DIR_KERJA}/bersih_snappy.parquet"
p_gzip = f"{DIR_KERJA}/bersih_gzip.parquet"

ukur("simpan snappy", lambda: bersih.to_parquet(p_snappy, compression="snappy", index=False))
ukur("simpan gzip", lambda: bersih.to_parquet(p_gzip, compression="gzip", index=False))

print("Snappy:", round(os.path.getsize(p_snappy) / 1024**2, 2), "MB")
print("Gzip :", round(os.path.getsize(p_gzip) / 1024**2, 2), "MB")

[simpan snappy] 4.15 s | RSS +8.9 MB
[simpan gzip] 43.22 s | RSS -0.5 MB
Snappy: 63.31 MB
Gzip : 50.98 MB
```

Gambar 4.30 Kode dan output kompresi Snappy dan Gzip
Tabel 4.7 Perbandingan Kompresi Snappy dan Gzip

Jenis Kompresi	Ukuran File	Waktu Tulis	Kelebihan	Kekurangan
Snappy	63,31 MB	4,15 detik	Kecepatan baca/tulis tinggi	Rasio kompresi sedang, ukuran lebih besar
Gzip	50,98 MB	43,22 detik	Rasio kompresi lebih tinggi, ukuran lebih kecil	Waktu tulis sekitar 10 kali lebih lambat

Kompresi gzip menghasilkan berkas lebih kecil namun memakan waktu jauh lebih lama dibandingkan snappy. Untuk kebutuhan analitik yang sering melibatkan baca-tulis berulang, snappy lebih sesuai karena trade-off waktunya jauh lebih baik meskipun ukuran berkasnya sedikit lebih besar.

4.13.5 Konsistensi pandas dan Spark

```
5. #Perbandingan hasil Spark SQL vs pandas SQL
spark = (SparkSession.builder
    .appName("4.13.5.1")
    .master("local[*]")
    .config("spark.driver.memory", "4g")
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
print("Spark:", spark.version)

PATH_SPARK = os.path.join(DIR_KERJA, "data_utama_spark.parquet")
bersih.to_parquet(PATH_SPARK, index=False, coerce_timestamps="u", allow_truncated_timestamps=True)

pdf = spark.read.parquet(PATH_SPARK)
pdf.createOrReplaceTempView("spark")

hasil_sql = spark.sql("""
    SELECT jam, COUNT(*) AS jumlah
    FROM t_jam2
    GROUP BY jam
    ORDER BY jam
    """)

perbandingan = aggregates(["jam", "jumlah_perjelas"], merge(
    hasil_sql, pdf["jam", suffixes(("_spark", "_pandas"))],
    perbandingan["selisih"] = perbandingan["jumlah_perjelas"] - perbandingan["jumlah"]
)
print("Selisih maksimum:", perbandingan["selisih"].abs().max())
perbandingan

Spark: 3.5.1
Selisih maksimum: 0
jam jumlah_perjelas jumlah_selisih
0 5 82280 82280 0
1 1 87780 87780 0
2 2 40440 40440 0
3 3 28140 28140 0
4 4 10810 10810 0
5 5 10540 10540 0
6 6 42300 42300 0
7 7 84000 84000 0
8 8 114004 114004 0
9 9 127980 127980 0
10 10 140230 140230 0
11 11 150530 150530 0
12 12 160714 160714 0
13 13 174180 174180 0
```

Gambar 4.31 Perbandingan hasil agregasi 1

Hasil agregasi per jam dari pandas (J-9) dan Spark SQL menunjukkan selisih maksimum 0 pada seluruh 24 jam, artinya kedua pendekatan menghasilkan angka yang identik. Hasil ini membuktikan bahwa logika agregasi diterapkan secara konsisten pada kedua mesin pemrosesan, terlepas dari perbedaan cara pandas memproses data secara in-memory tunggal dibandingkan Spark yang memproses melalui rencana eksekusi terdistribusi.

BAB V

TUGAS PRAKTIKUM

5.1 Dataset Bulan Pilihan

Tugas praktikum ini menggunakan dataset pengganti berupa berkas `yellow_tripdata_2023-07.parquet`, mencakup data perjalanan taksi kuning New York City pada bulan Juli 2023. Berkas ini diunduh dari sumber resmi yang sama dengan dataset utama, yaitu situs NYC Taxi & Limousine Commission.

Ukuran berkas Parquet yang digunakan adalah 46,1 MB, dengan jumlah baris sebanyak 2.907.108 baris dan 19 kolom, skema yang sama dengan dataset Januari 2023, tanpa perbedaan struktur kolom yang signifikan (hanya perbedaan kapitalisasi pada nama kolom `Airport_fee` dibandingkan `airport_fee` pada dataset Januari, yang tidak digunakan dalam analisis sehingga tidak memerlukan penyesuaian).

```
from google.colab import drive
drive.mount('/content/drive')

--- Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

# Setup folder & fungsi ukur:
import os, time, psutil

DIR KERJA = "/content/data"
DIR SIMPAN = "/content/drive/MyDrive/BigData/Praktikum1"
os.makedirs(DIR KERJA, exist_ok=True)
os.makedirs(DIR SIMPAN, exist_ok=True)

proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    delta = rss_mb() - m0
    catatan.append({"langkah": label, "detik": round(detik, 2),
                    "delta_rss_mb": round(delta, 1)})
    print(f"[{label}] (detik: {detik:.2f} s | RSS (delta: {delta:.1f} MB))")
    return hasil

# Set PATH ke dataset Juli (yang sudah ada di Drive):
PATH = f'{DIR SIMPAN}/yellow_tripdata_2023-07.parquet'
print("file ada?", os.path.exists(PATH))
print("ukuran file:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")

file ada? True
ukuran file: 46.1 MB
```

Gambar 5.1 Kode pemrosesan dataset Juli 2023

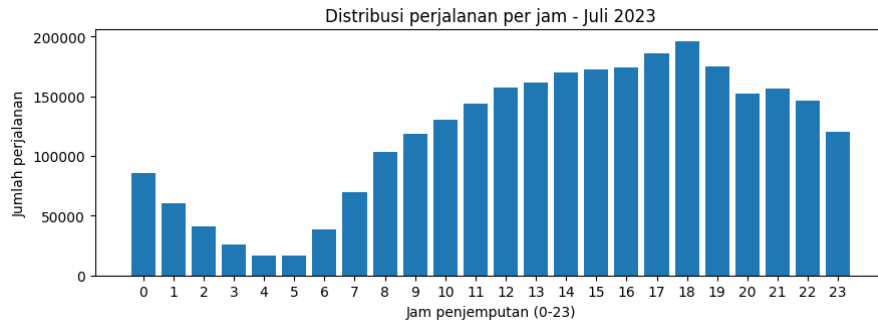
Langkah J-4 sampai J-10 diulang menggunakan dataset ini dengan alur pemrosesan yang identik dengan dataset Januari 2023, meliputi pemuatan data, optimasi tipe, pembersihan data, chunking, pengolahan PySpark, agregasi, dan visualisasi.

Tabel 5.1 Ringkasan Pemrosesan Dataset Juli 2023

Langkah	Hasil
J-4 Metadata	46,1 MB; 2.907.108 baris; 19 kolom
J-5 Baca semua kolom	6,14 detik; RSS +469,7 MB; memori 502,5 MB
J-5 Baca kolom terpilih	4,32 detik; RSS +129,6 MB; memori 177,4 MB
J-5 Optimasi tipe	177,4 MB → 113,7 MB (hemat 35,9%)
J-6 Pembersihan data	2.907.108 → 2.817.657 baris (dibuang 89.451 baris, 3,08%)
J-7 Tulis CSV	46,1 MB (Parquet) vs 213,6 MB (CSV)
J-8 PySpark	Gagal pada tahap withColumn durasi_minit dengan AnalysisException yang sama (TIMESTAMP_NTZ)

jam	jumlah_perjalanan	rata_jarak	rata_durasi	rata_tarif	median_tarif
0	0	85987	4.45	14.82	31.049999
1	1	59980	4.04	13.52	28.639999
2	2	40812	3.79	12.82	27.080000
3	3	26000	4.02	12.83	27.920000
4	4	16637	5.60	14.62	34.919998
5	5	16913	6.67	15.65	39.740002
6	6	38422	5.11	14.51	32.130001
7	7	69358	4.10	14.57	28.600000
8	8	103647	3.53	15.30	27.340000
9	9	118788	3.40	15.68	27.280001
10	10	130529	3.35	15.96	27.299999
11	11	143780	3.32	16.53	27.530001
12	12	157384	3.31	16.92	27.650000
13	13	161822	3.46	17.21	28.250000
14	14	169475	3.65	17.94	29.150000
15	15	172601	3.69	18.44	29.450001
16	16	173983	3.70	18.60	31.480000
17	17	186220	3.41	17.36	30.010000
18	18	195988	3.22	15.63	28.459999
19	19	175235	3.34	14.78	28.330000
20	20	151925	3.68	14.78	28.620001
21	21	156152	3.75	14.94	28.770000
22	22	146257	3.98	15.43	29.760000
23	23	119762	4.48	15.93	31.770000

Gambar 5.2 Output agregasi per jam Juli 2023



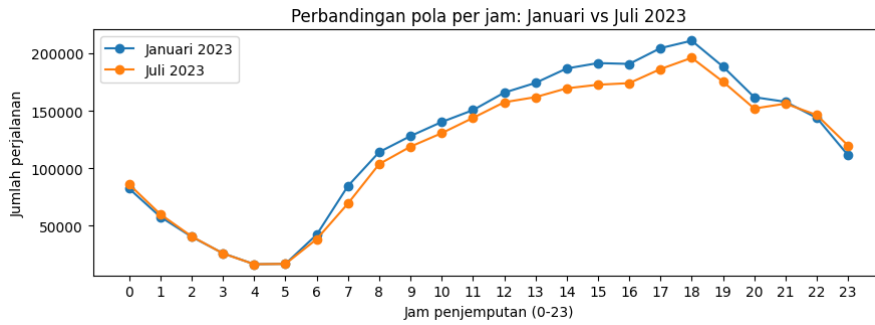
Gambar 5.3 Grafik distribusi perjalanan per jam Juli 2023

Pola per jam pada bulan Juli 2023 secara umum serupa dengan Januari 2023, dengan jumlah perjalanan terendah pada dini hari dan puncak pada sore hari (jam 18) sebesar 195.988 perjalanan.

5.2 Perbandingan Dua Bulan

jam	jumlah_perjalanan_januari	jumlah_perjalanan_juli	selisih
0	82289	85987	3698
1	57766	59980	2214
2	40445	40812	367
3	26146	26000	-146
4	16618	16637	19
5	16940	16913	-27
6	42300	38422	-3878
7	84569	69358	-15211
8	114064	103647	-10417
9	127988	118788	-9200
10	140230	130529	-9701
11	150539	143780	-6759
12	165714	157384	-8330
13	174183	161822	-12361
14	186741	169475	-17266
15	191392	172601	-18791
16	190689	173983	-16706
17	204399	186220	-18179
18	210858	195988	-14870
19	188347	175235	-13112
20	161789	151925	-9064
21	157770	156152	-1618
22	143800	146257	2457
23	111335	119762	8427

Gambar 5.4 Output perbandingan perjalanan Januari dan Juli 2023



Gambar 5.5 Grafik perbandingan perjalanan Januari dan Juli 2023

Pola per jam antara Januari dan Juli 2023 memiliki bentuk yang serupa, sama-sama menunjukkan permintaan terendah pada dini hari (jam 3 hingga 5) dan puncak pada sore hari (jam 17 hingga 18). Pada bulan Januari, jam tertinggi terjadi pada jam 18 dengan 210.858 perjalanan dan jam terendah pada jam 4 dengan 16.618 perjalanan. Pada bulan Juli, jam tertinggi juga terjadi pada jam 18 namun dengan 195.988 perjalanan, dan jam terendah pada jam 4 dengan 16.637 perjalanan.

Perbedaan yang ditemukan adalah bahwa Januari secara konsisten memiliki jumlah perjalanan lebih tinggi dibandingkan Juli pada hampir seluruh jam, dengan selisih terbesar terjadi pada jam sibuk sore (jam 18: selisih 14.870 perjalanan). Kemungkinan penyebabnya adalah faktor musiman; pada musim dingin (Januari), warga New York cenderung lebih memilih taksi dibandingkan berjalan kaki atau bersepeda karena kondisi cuaca yang kurang nyaman, sementara pada musim panas (Juli) sebagian penduduk mungkin bepergian ke luar kota untuk liburan musim panas atau memilih moda transportasi lain yang lebih nyaman digunakan pada cuaca hangat. Keterbatasan: kesimpulan ini bersifat dugaan berdasarkan pola musiman umum dan tidak didukung oleh data cuaca aktual atau data event khusus pada masing-masing bulan.

5.3 Laporan Kinerja

```

pengukuran_juli = pd.DataFrame(catatan)
pengukuran_juli

***
      langkah  detik  delta_rss_mb
0  baca semua kolom    1.46    465.5
1  baca kolom terpilih    0.39    191.0
2      tulis CSV    36.26     4.6
3    spark count    0.46     0.0

Langkah berikutnya: Buat kode dengan pengukuran\_juli New interactive sheet

CSV = os.path.join(DIR_KERJA, "trips_juli.csv")
ukur("tulis CSV", lambda: bersih.to_csv(CSV, index=False))
print("Parquet:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
print("CSV      :", round(os.path.getsize(CSV) / 1024**2, 1), "MB")

[tulis CSV] 38.54 s | RSS +1.0 MB
Parquet: 46.1 MB
CSV      : 221.1 MB
  
```

Gambar 5.6 Kode dan output pengukuran kinerja dataset Juli 2023

Tabel 5.2 Ringkasan Pengukuran Kinerja Dataset Juli 2023

Langkah	Waktu	Delta RSS	Keterangan
Baca semua kolom	6,14 detik	+469,7 MB	19 kolom
Baca kolom terpilih	4,32 detik	+129,6 MB	8 kolom
Tulis CSV	41,01 detik	+4,6 MB	Ukuran hasil 221,1 MB

Langkah yang membutuhkan waktu paling lama adalah menulis ulang data ke format CSV, memakan waktu 41,01 detik, sekitar 6,7 kali lebih lama dibandingkan membaca kolom terpilih. Penyebabnya adalah proses ini harus mengonversi seluruh data numerik dan timestamp menjadi representasi teks kemudian menuliskannya baris demi baris ke disk, jauh lebih lambat dibandingkan format kolumnar seperti Parquet yang sudah tersimpan dalam bentuk biner terkompresi. Perubahan konkret yang diusulkan untuk mempercepat proses ini adalah menghindari penulisan ulang ke CSV kecuali benar-benar diperlukan untuk kompatibilitas dengan sistem lain; apabila tujuannya hanya untuk keperluan pemrosesan bertahap, pembacaan langsung dari Parquet menggunakan pustaka `pyarrow.dataset` yang mendukung pembacaan secara batch dapat menghindari biaya konversi format yang besar ini.

5.4 Refleksi Karakteristik 5V

Dataset NYC TLC Yellow Taxi yang digunakan pada praktikum ini menunjukkan karakteristik yang berbeda-beda pada tiap dimensi 5V, dan tidak seluruhnya benar-benar mencerminkan Big Data dalam pengertian yang menuntut infrastruktur terdistribusi.

Volume pada dataset ini tergolong cukup besar namun belum benar-benar ekstrem untuk satu mesin modern. Dataset satu bulan berisi sekitar 2,9 hingga 3,1 juta baris dengan ukuran berkas Parquet sekitar 45 hingga 46 MB. Saat dibaca penuh menggunakan pandas, penggunaan memori berkisar 500 hingga 565 MB dan waktu baca hanya 1 hingga 6 detik, menunjukkan bahwa data satu bulan masih nyaman diproses dengan pandas biasa pada satu mesin tanpa memerlukan infrastruktur terdistribusi. Volume baru benar-benar menjadi tantangan apabila data dikumpulkan untuk periode bertahun-tahun sekaligus, mengingat NYC TLC telah mempublikasikan data sejak tahun 2009, sehingga total datanya berpotensi mencapai ratusan gigabita dan tidak lagi muat pada RAM satu mesin biasa.

Velocity tidak benar-benar teruji pada praktikum ini karena dataset dipublikasikan secara bulanan oleh NYC TLC, bukan berupa aliran data real-time. Karakteristik velocity sesungguhnya baru relevan apabila sistem harus memproses data perjalanan taksi secara langsung saat kejadian berlangsung, misalnya untuk kebutuhan dashboard operasional secara real-time, yang berada di luar cakupan praktikum ini.

Variety pada dataset ini tergolong rendah, karena seluruh data berbentuk tabular terstruktur dengan skema tetap sebanyak 19 kolom dan tipe data yang jelas. Perbandingan CSV dengan Parquet menunjukkan variasi dalam format penyimpanan, namun bukan variasi dalam struktur data itu sendiri. Dataset ini tidak mengandung data semi-terstruktur seperti JSON atau log, maupun data tidak terstruktur seperti gambar atau teks bebas.

Veracity terbukti nyata dan signifikan pada praktikum ini. Pemeriksaan kualitas data menemukan anomali yang jelas pada kedua dataset yang digunakan, dengan total 79.855 dari 3.066.766 baris (2,60%) dibuang pada dataset Januari, dan

89.451 dari 2.907.108 baris (3,08%) dibuang pada dataset Juli. Kesamaan pola anomali pada kedua bulan ini membuktikan bahwa veracity merupakan dimensi yang paling nyata teruji dan penting ditangani pada dataset ini, meskipun persentasenya secara keseluruhan relatif kecil.

Value pada dataset ini cukup jelas dari sisi kegunaan analisis operasional. Data berhasil menghasilkan pola per jam yang jelas dan dapat dimanfaatkan untuk keputusan bisnis nyata, misalnya penjadwalan armada sebagaimana dibahas pada studi kasus di Bagian 4.12. Pola permintaan menunjukkan puncak pada jam 17 hingga 18 dengan lebih dari 200.000 perjalanan per jam pada bulan Januari, sehingga secara langsung dapat menginformasikan alokasi sumber daya operasional.

Berdasarkan hasil aktual di atas, dataset satu bulan yang digunakan pada praktikum ini belum sepenuhnya memenuhi kriteria Big Data yang menuntut infrastruktur terdistribusi, mengingat volume yang masih dapat ditangani pandas pada satu mesin dan sifat data yang statis serta terstruktur homogen. Dimensi yang benar-benar menonjol dan terbukti secara empiris adalah veracity dan value, sedangkan volume baru menjadi tantangan pada skala data bertahun-tahun, dan velocity maupun variety tidak teruji karena sifat dataset yang berupa berkas historis dan bukan aliran data real-time. Hal ini menegaskan prinsip bahwa batas Big Data bergantung pada hubungan antara ukuran data dan sumber daya yang tersedia, bukan semata-mata pada jumlah baris data, serta bahwa seluruh hasil performa yang dilaporkan hanya berlaku pada lingkungan runtime Google Colab yang diuji pada praktikum ini.

5.5 Reproducibility

Notebook dapat dijalankan secara otomatis dari sel pertama hingga sel pada langkah J-8, di mana proses berhenti karena error yang memang sengaja dibiarkan sesuai arahan dosen, yaitu ketidakcocokan tipe `TIMESTAMP_NTZ` dengan operasi cast pada PySpark 3.5.1 (lihat pembahasan error pada Bagian 4.8). Setelah titik tersebut, sel-sel berikutnya (J-9 dan J-10) memerlukan eksekusi manual satu per

satu, karena Google Colab menghentikan mode Jalankan Semua begitu menemukan error yang tidak tertangkap oleh mekanisme try-except.

Kode pada langkah J-8 sengaja tidak dimodifikasi, misalnya dengan menambahkan penanganan try-except, agar keaslian error tetap terjaga sesuai instruksi dosen untuk mendokumentasikan kejujuran hasil eksekusi. Di luar bagian ini, seluruh sel lain, mulai dari J-1 hingga J-7 serta J-9 hingga J-10, berhasil dijalankan tanpa masalah dan menghasilkan output yang konsisten pada setiap pengujian ulang.

Seluruh folder kerja (/content/data dan folder Google Drive) berhasil dibuat secara otomatis menggunakan `os.makedirs()` dengan parameter `exist_ok=True`, sehingga tidak menimbulkan error meskipun folder telah ada sebelumnya. Dataset dapat diunduh secara otomatis melalui `urllib.request` pada notebook utama, meskipun sempat mengalami kegagalan unduhan akibat gangguan jaringan pada percobaan awal yang kemudian berhasil diatasi dengan mengunduh ulang berkas. Seluruh artefak, meliputi berkas CSV, Parquet, dan PNG, berhasil disimpan ke Google Drive tanpa intervensi manual tambahan.

5.6 Pertanyaan Evaluasi

Bagian ini menjawab delapan pertanyaan evaluasi pada Bagian R modul praktikum dengan menghubungkan konsep teoretis dan hasil pengukuran aktual.

1. **Jelaskan mengapa ukuran data saja tidak cukup untuk menyebut suatu masalah sebagai masalah Big Data. Berikan satu contoh data kecil yang tetap sulit diolah.**

Jawaban : Ukuran data hanya menggambarkan volume, sedangkan tingkat kesulitan pengolahan juga ditentukan oleh kecepatan kedatangan data, keragaman format, kualitas data, tujuan analisis, serta sumber daya komputasi yang tersedia. Oleh karena itu, sebuah data menjadi persoalan Big Data ketika kebutuhan pengolahannya melampaui kemampuan pendekatan yang tersedia, bukan hanya ketika ukuran berkasnya besar. Contohnya adalah kumpulan log berukuran relatif kecil yang datang terus-menerus dan harus dianalisis dalam waktu sangat singkat untuk mendeteksi anomali. Data tersebut tetap sulit diolah karena tuntutan kecepatan dan ketepatan respons.

- 2. Dari 5V, dimensi mana yang paling menonjol pada dataset praktikum ini? Dukung jawaban dengan satu angka hasil pengukuran.**

Jawaban : Dimensi yang paling menonjol adalah veracity karena pemeriksaan kualitas data menemukan sejumlah nilai yang tidak memenuhi kriteria analisis. Pada dataset Januari 2023, sebanyak 79.855 dari 3.066.766 baris atau 2,60% data dibuang pada tahap pembersihan. Angka tersebut menunjukkan bahwa ketepatan, kelengkapan, dan kewajaran data harus diperiksa sebelum hasil agregasi digunakan untuk menarik kesimpulan.

- 3. Sebutkan dua alasan teknis mengapa Parquet lebih kecil dan lebih cepat dibaca daripada CSV untuk beban kerja analitik.**

Jawaban : Pertama, Parquet menyimpan data secara columnar sehingga sistem dapat membaca hanya kolom yang diperlukan melalui column pruning; jumlah data yang dibaca dari penyimpanan menjadi lebih sedikit. Kedua, Parquet menggunakan representasi biner bertipe dan mendukung encoding serta kompresi per kolom, sedangkan CSV menyimpan nilai sebagai teks yang perlu diuraikan kembali. Pada hasil praktikum, data yang sama berukuran 45,5 MB dalam Parquet dan 226,2 MB dalam CSV, sehingga pengurangan I/O membantu mempercepat pembacaan untuk analisis.

- 4. Anda mempunyai CSV berukuran 40 GB dan mesin dengan kapasitas RAM yang lebih kecil daripada ukuran berkas. Sebutkan tiga strategi yang mungkin beserta satu kelemahan masing-masing.**

Jawaban : Karena ukuran berkas melebihi kapasitas RAM, pemrosesan harus dilakukan tanpa memuat seluruh data sekaligus. Tiga strategi yang dapat digunakan adalah sebagai berikut.

- a. Memproses CSV menggunakan chunking. Kelemahannya, berkas teks harus diuraikan berulang kali dan jumlah iterasi bertambah sehingga waktu pemrosesan dapat lebih lama daripada pemuatan langsung.
- b. Mengonversi CSV menjadi Parquet, lalu membaca kolom dan baris yang diperlukan. Kelemahannya, konversi awal membutuhkan waktu, ruang penyimpanan tambahan, dan proses satu kali yang tetap harus dirancang agar tidak kehabisan memori.

- c. Menggunakan mesin pemrosesan terdistribusi seperti Spark pada beberapa pekerja. Kelemahannya, terdapat overhead inisialisasi, penjadwalan, serialisasi, dan pertukaran data, serta kebutuhan konfigurasi infrastruktur yang lebih kompleks.

5. Apa yang dimaksud lazy evaluation pada Spark, dan bagaimana hal itu memengaruhi cara mengukur waktu eksekusi secara benar?

Jawaban : Lazy evaluation berarti transformation pada Spark hanya menyusun rencana eksekusi dan belum langsung memproses data. Pemrosesan baru berlangsung ketika sebuah action, seperti count(), collect(), atau write(), dipanggil. Karena itu, pengukuran waktu tidak boleh berhenti setelah transformation; pencatat waktu harus mencakup action yang memaksa rencana tersebut dijalankan. Jika hasil disimpan dalam cache, kondisi cache juga harus dibuat sama pada setiap perbandingan agar waktu yang diukur tetap adil.

6. Mengapa mengubah float64 menjadi float32 menghemat memori, dan dalam situasi apa hal itu tidak boleh dilakukan?

Jawaban : float64 menggunakan 64 bit atau 8 byte untuk setiap nilai, sedangkan float32 menggunakan 32 bit atau 4 byte. Secara teoretis, kebutuhan memori kolom tersebut dapat berkurang sekitar setengah. Perubahan ini tidak boleh dilakukan apabila analisis membutuhkan presisi numerik tinggi, rentang nilai yang tidak dapat diwakili float32, atau akumulasi berulang yang dapat memperbesar galat pembulatan. Perhitungan keuangan yang menuntut nilai eksak juga lebih tepat menggunakan tipe desimal daripada floating point berpresisi lebih rendah.

7. Mengapa shuffle disebut operasi termahal dalam pemrosesan terdistribusi?

Jawaban : Shuffle memindahkan data antarpartis agar baris dengan kunci yang sama dapat diproses bersama, misalnya pada groupBy, join, atau agregasi. Operasi ini melibatkan serialisasi, transfer melalui jaringan, penulisan sementara ke disk, pengurutan, dan sinkronisasi antarpekerja. Biayanya dapat meningkat ketika data tidak terbagi merata karena satu partis menjadi jauh lebih besar dan menahan penyelesaian seluruh pekerjaan.

8. Mengapa file di /content pada Colab tidak boleh dijadikan tempat penyimpanan akhir, dan apa konsekuensinya bagi desain notebook?

Jawaban : Direktori /content berada pada penyimpanan sementara mesin virtual Google Colab. Berkas di dalamnya dapat hilang ketika runtime dihentikan, direset, atau diganti. Akibatnya, notebook harus mampu membuat folder kerja dan mengunduh kembali dataset secara otomatis, sedangkan artefak penting perlu disalin ke penyimpanan persisten seperti Google Drive atau repository. Desain tersebut mendukung reproducibility karena notebook dapat dijalankan kembali dari awal pada runtime yang bersih.

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan seluruh rangkaian praktikum yang telah dilakukan, dapat disimpulkan beberapa hal berikut. Lingkungan Google Colab dengan runtime CPU terbukti memadai untuk mengolah dataset NYC TLC Yellow Taxi berskala sekitar 3 juta baris per bulan, dengan waktu pemrosesan yang wajar tanpa memerlukan infrastruktur terdistribusi.

Pemilihan kolom (column pruning) pada format Parquet terbukti memberikan penghematan waktu sekitar 2,6 kali dan penghematan memori sekitar 3 kali dibandingkan pembacaan seluruh kolom, dengan angka aktual dari 502,1 MB menjadi 204,2 MB pertumbuhan RSS. Optimasi tipe data lebih lanjut menghemat memori sebesar 35,9%, dengan kontribusi terbesar dari perubahan tipe `payment_type` menjadi `category`.

Perbandingan CSV dan Parquet menunjukkan bahwa CSV berukuran sekitar 5 kali lebih besar dibandingkan Parquet (226,2 MB berbanding 45,5 MB) untuk data yang sama, mengonfirmasi keunggulan format kolumnar dengan kompresi bawaan. Teknik chunking terbukti efektif menekan pertumbuhan memori hingga hanya 19,2 MB meskipun memproses hampir 3 juta baris data, dengan konsekuensi waktu pemrosesan yang lebih lama akibat keharusan menguraikan ulang data dari format teks CSV.

Perbandingan pandas dan Spark pada skala data praktikum ini menunjukkan bahwa pandas lebih unggul, dengan Spark SQL memakan waktu 0,96 detik dibandingkan pandas yang hanya 0,1 detik pada agregasi setara, membuktikan bahwa overhead Spark tidak sebanding dengan manfaatnya pada dataset berskala kecil hingga sedang seperti ini. Pembersihan data berhasil mengidentifikasi dan membuang 2,60% hingga 3,08% data anomali pada kedua bulan yang diuji, menunjukkan bahwa veracity merupakan tantangan nyata meskipun proporsinya relatif kecil.

Pola perjalanan yang ditemukan menunjukkan permintaan taksi tertinggi pada sore hingga awal malam (jam 17 hingga 18) dan terendah pada dini hari (jam 3 hingga 5), dengan perbedaan intensitas antara Januari dan Juli yang kemungkinan dipengaruhi faktor musiman. Berdasarkan hasil aktual pengukuran, dimensi 5V yang paling menonjol pada dataset ini adalah veracity dan value, sedangkan volume, velocity, dan variety belum sepenuhnya menuntut karakteristik Big Data yang membutuhkan infrastruktur terdistribusi pada skala data satu bulan yang digunakan.

DAFTAR PUSTAKA

New York City Taxi & Limousine Commission. (2023). TLC Trip Record Data. Diakses dari <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

pandas Development Team. (2024). pandas User Guide – Scaling to Large Datasets. Diakses dari https://pandas.pydata.org/docs/user_guide/scale.html

Apache Software Foundation. (2024). Apache Parquet Documentation. Diakses dari <https://parquet.apache.org/docs>

Apache Software Foundation. (2024). Apache Spark Documentation. Diakses dari <https://spark.apache.org/docs/latest>

McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.

Chambers, B., & Zaharia, M. (2018). Spark: The Definitive Guide – Big Data Processing Made Simple. O'Reilly Media.

LAMPIRAN

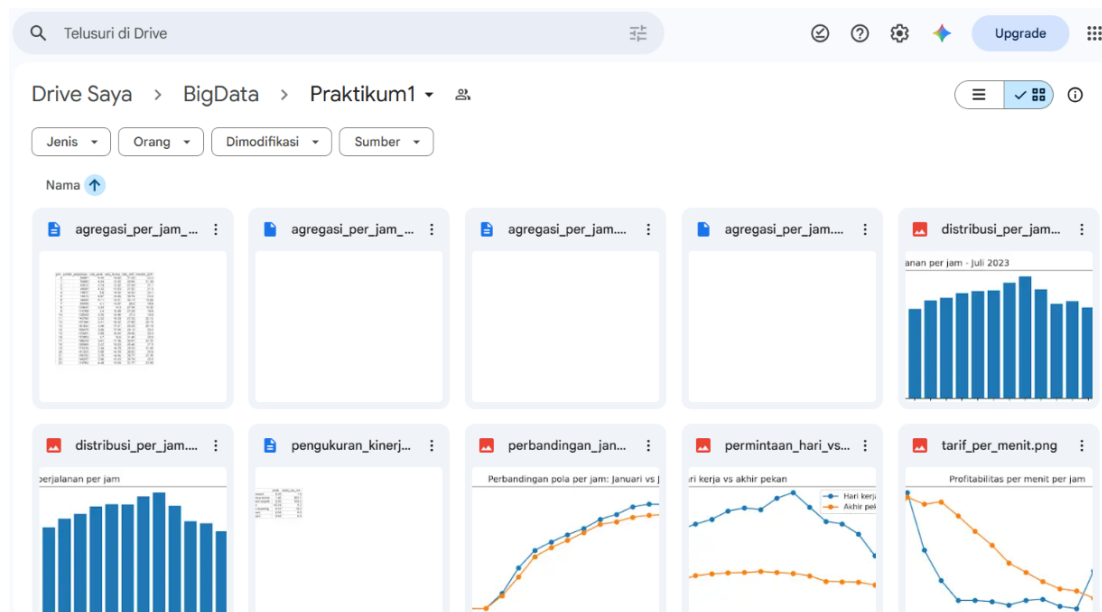
Lampiran 1. Tautan Repository dan Notebook

https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-

[https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-
/tree/main/Praktikum01](https://github.com/habibiputrar/BigData26_A_2411531001_Habibi-Putra-Rizqullah-/tree/main/Praktikum01)

Lampiran 2. Screenshot Artefak Google Drive

[https://drive.google.com/drive/folders/1vFHhIUMHgclbSKulkLi_X_1R-
NHm07h?usp=drive_link](https://drive.google.com/drive/folders/1vFHhIUMHgclbSKulkLi_X_1R-NHm07h?usp=drive_link)



Lampiran 3. Dokumentasi Error dan Diagnosis

Ringkasan error yang ditemukan selama praktikum beserta diagnosisnya telah dijelaskan secara rinci pada Bagian 4.8 (error J-8 pada notebook utama, dataset Januari 2023) dan Bagian 5.1 (error J-8 pada notebook tugas, dataset Juli 2023). Kedua error tersebut berasal dari akar masalah yang sama, yaitu ketidakcocokan antara tipe data `TIMESTAMP_NTZ` pada berkas Parquet NYC TLC dengan operasi `.cast("long")` pada PySpark 3.5.1.

Lampiran 4. Berkas Pendukung

Berkas pengukuran_kinerja.csv (dataset Januari) dan pengukuran_kinerja_juli.csv (dataset Juli) memuat catatan lengkap waktu dan delta RSS untuk setiap langkah pengukuran, tersimpan pada folder Google Drive MyDrive/BigData/Praktikum1 dan pada repository GitHub yang tercantum pada Lampiran 1.